RESEARCH WHITE PAPER · VERSION 2.0

Verified Compute Fabric

A reference architecture for evidence-bound authority and breach containment

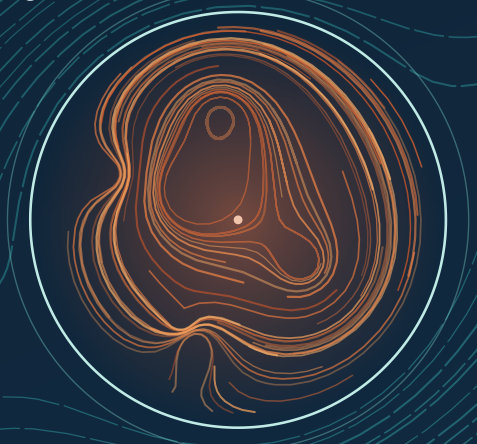
ARCHITECTURAL THESIS

A workload's authority should come from authenticated evidence and a current, resource-specific policy. Compromise must not give the workload control over the boundaries that contain it.

Co-Authored with Human Assistance from: **Gabe Fletcher**

24 October 2026

Internet infrastructure | Software systems | Hardware trust | AI agents



STATUS: PROPOSED REFERENCE ARCHITECTURE

Research-backed design synthesis. No integrated implementation, benchmark result or certification is claimed.

85 primary-source references · 12 figures · two deployment profiles

Version 1.0 drafted with ChatGPT (OpenAI). Version 2.0 revised and extended with Claude (Anthropic).

Cover: Bounded Weave, seed 2026. Threads of authority, issued as short leases, flow around a contained breach. None crosses the boundary.

ABOUT THIS PAPER

What this paper is, and what it is not

Co-Authored with Human Assistance from: **Gabe Fletcher**

Verified Compute Fabric is a reference architecture. It takes security mechanisms that already exist — zero-trust access control, remote attestation, workload identity, supply-chain provenance, confidential computing — and defines the contract that makes them work together. It then says how to test that contract. The standards it cites support individual properties. They do not establish the security of the integrated design described here.

“Verified” has one meaning in this paper: a named verifier has authenticated evidence and evaluated named claims against a versioned policy. It is not a claim of formal verification, universal safety or certification. The name does not designate an industry standard.

THIS PAPER PROVIDES

- Design rules and a reference composition of control planes.
- An exposure model that ties each control to the harm it limits.
- Analytical bounds on revocation, with every assumption stated.
- Requirements traceable to controls and to acceptance tests.
- An adoption path for a small Docker-based estate.

THIS PAPER DOES NOT CLAIM

- An implementation, a benchmark or a certification.
- That authorized software is harmless.
- That data already disclosed can be recalled.
- Availability under resource exhaustion or denial of service.
- Protection from physical attacks on memory, unless declared.

Figures and numbers marked *illustrative* use stated parameters, not measurements. Every real-world date, figure and incident was checked against a primary source on 24 October 2026.

What changed in version 2.0

- **An exposure model.** Harm is bounded by reach, rate, budget and time window (§1). Every control now maps to one of those terms.
- **A tighter revocation bound.** Delegated authority may not outlive the authority that issued it (“nested expiry”). This takes permit lifetime out of the worst case, so permits can be cached longer without weakening revocation (§15).
- **AI agents as workloads.** A new section treats agents as programs whose control flow can be steered by their inputs (§9).
- **Transparency and cryptographic agility.** Release approvals go into an append-only log (§5), and long-lived roots get a post-quantum migration plan (§12).
- **Evidence from real incidents.** Seven public incidents anchor the boundaries (§1). Recent memory-interposer attacks sharpen the hardware claims (§11).
- **Figures, tests and traceability.** 12 figures, nine new acceptance tests and five new requirements (R-13 to R-17). Cautions are stated once, here. Appendix B has the full change log.

CONTENTS

Reading map

PART I · THE PROBLEM AND THE MODEL

01 Why breaches spread: the exposure model 5

02 Scope, evidence and threat model 7

PART II · ARCHITECTURE

03 Control planes and trust boundaries 8

04 The evidence-bound lifecycle 9

PART III · MECHANISMS

05 Software supply chain and transparency 11

06 Identity, credentials and delegation 12

07 Attestation and launch binding 14

08 Runtime, data and operation budgets 16

09 AI agents as workloads 17

10 Internet infrastructure 19

11 Hardware and the protected boundary 20

12 Cryptographic agility and post-quantum migration 21

PART IV · CONTRACTS

13 Interoperability conventions 22

14 The authorization contract 24

15 Revocation and nested expiry 25

16 Failure semantics 27

17 Governance and recovery 29

PART V · OPERATIONS

18 Deployment profiles 30

19 A Docker and Coolify adoption path 31

20 Adoption gates and economics 33

21 Validation: acceptance tests 34

22 Measurement, limitations and research agenda 36

23 Requirement traceability 37

24 Conclusion 38

BACK MATTER

A Terms used in this paper 39

B Changes from version 1.0 40

References 41

EXECUTIVE BRIEF

Limit what a breach can do

Large breaches rarely need a large first foothold. They need authority that is broad, long-lived and controlled by the thing that was compromised.

Think of a service key that can read every tenant, a token that works from any network for months, or an admin path nobody watches. The first mistake is small. The authority behind it is not. Public reviews of SolarWinds, Storm-0558 and the Snowflake intrusions show the same pattern. The way in differed every time. The damage came from what the first foothold could reach.

Verified Compute Fabric binds every protected operation to authenticated evidence about the running workload and to a current, resource-specific policy. It issues authority as short leases. Components the workload does not control enforce them. Authorities kept out of an attacker's reach revoke or rebuild them.

The goal is a damage bound you can compute and test: what a compromised workload can reach, how fast it can act and for how long. On the illustrative numbers in Figure 1, exposure falls from about 2,700 tenant-hours to about 70 when the attacker runs inside the workload. For a stolen token it falls to almost nothing.

Five design rules

1. **Prove what runs.** Approve exact code and launch configuration, record the approval in a public log, and bind evidence of that release to the running instance.
2. **Grant per operation.** Authority names a resource, an action, a tenant and an audience. Identity alone grants nothing.
3. **Make every grant expire.** Leases are short. Renewal needs fresh evidence. No grant outlives the authority that issued it.
4. **Enforce from outside.** Decisions are checked where the workload cannot reach: a gateway, a database policy, a protected-guest supervisor, an off-host broker.
5. **Keep recovery out of reach.** Restoring trust must work when production identity, policy and administrator accounts are gone.

Deployment decision

The **managed-host profile** trusts the host administrator and kernel, and contains application compromise with external identity and resource-level authorization. The **host-adversarial profile** adds a supported confidential execution environment, an attested guest supervisor and an independent verifier. Its guarantees depend on the hardware vendor's security model and the protected guest's code. Neither profile makes authorized software harmless. Neither recalls data already disclosed.

Where to start

Pick one sensitive operation, such as a tenant data export. Put it behind an independent gateway with a 60-second lease, sender-bound tokens, a hard export budget and a tested revocation path. Build releases on a separate host, deploy only exact digests, and measure the result. Expand only when the operation passes its gate (§20). Record every control you have not yet built as an assurance gap.

01 / THE PROBLEM

Breaches spread through authority, not through the first bug

Public incident reviews tell a consistent story. The way in varies: a stolen password, a poisoned build, a forged token. What turns it into a crisis is the authority waiting on the other side.

The table lists seven well-documented incidents and the boundary each one shows to be missing or too weak. None of this is a claim that the architecture in this paper would have prevented the incident. Each case shows which limit mattered.

Incident	What happened	Boundary it shows
SolarWinds, 2020	An implant on the vendor's build server swapped a source file during compilation, so the vendor's own signed updates carried a backdoor. [1]	Isolated, verifiable builds with provenance (§5)
Storm-0558, 2023	Tokens signed with a 2016 consumer signing key, meant to be retired in 2021, were accepted for enterprise mailboxes because of a validation flaw. Twenty-two organizations and over 500 individuals were affected. [2]	Scoped issuers whose authority expires; resources reject out-of-scope issuers (§6)
Snowflake customers, 2024	Credentials stolen by infostealer malware, some unrotated for up to four years, were used against accounts with no second factor and no network allow-list. About 165 organizations were notified. [3]	Short-lived, sender-bound credentials; phishing-resistant login (§6)
xz-utils, 2024	A maintainer identity built up over more than two years published release tarballs containing a build trigger that was absent from the public repository. It activated a payload hidden in committed test files. It was disclosed before wide deployment. [4]	Build from reviewed source; runtime limits still apply to signed software (§5, §8)
CrowdStrike, 2024	A faulty security-content update went to Windows hosts all at once and was pulled 78 minutes later. A template expected 21 input fields and received 20. About 8.5 million devices crashed. [5, 6]	The control plane needs its own blast radius: staged rollout and canaries (§16)
tj-actions, 2025	Version tags of a GitHub Action used by over 23,000 repositories were moved to a malicious commit that printed CI secrets into build logs. [7]	Pin by digest; short-lived, scoped build secrets (§5)
Shai-Hulud, 2025	A self-replicating npm worm stole developer and cloud tokens and used them to publish infected versions of more than 500 packages. [8]	Publishing without long-lived tokens (§5)

The exposure model

Treat compromise as a given and ask how much harm it can do before it is stopped. For an attacker who holds authority a, the harm is bounded by

$$E(a) \le \sum_{r \in \mathrm{Reach}(a)} \min\left(B_r,\ \rho_r \cdot W\right), \qquad W = T_{\mathrm{detect}} + T_{\mathrm{decide}} + T_{\mathrm{stop}}$$

$\mathrm{Reach}(a)$ is the set of resources and actions the authority can exercise. ρ_r is the fastest rate at which the attacker can act on resource r. B_r is a hard budget on r, such as a cap on rows exported per lease. W is the window from compromise until the authority stops working.

Each design rule shrinks one term. Per-operation grants and tenant binding shrink Reach. Rate limits and operation budgets cap ρ and B. Short leases, nested expiry and push revocation shrink T_{stop}. Detection time is outside this paper's controls, but it sits inside W, so it must be measured.

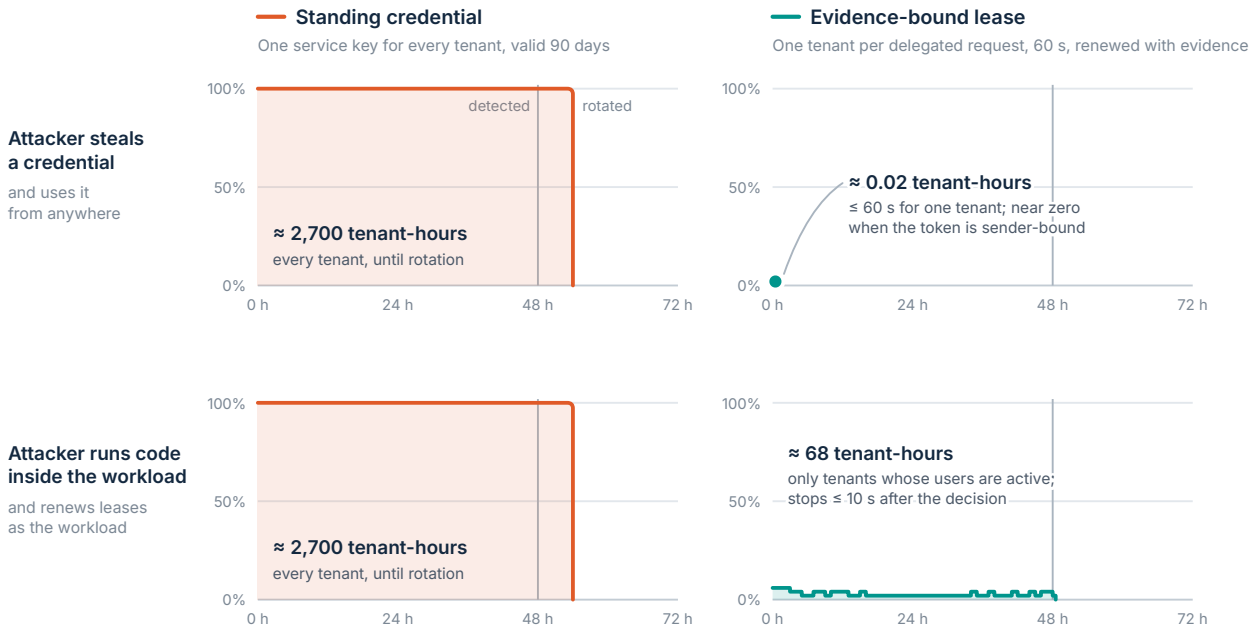


Figure 1. Exposure under two credential designs and two kinds of attacker. The shaded area is exposure in tenant-hours of reachable data. *Illustrative model:* 50 tenants; compromise detected at 48 h in every panel; a standing key takes 6 h to rotate; a lease revocation takes effect within 10 s of a 30-minute decision; under leases a compromised workload can act only for tenants whose users are active, one to six at a time.

Two attackers, two levers. The model separates two cases that are often blurred. An attacker who *steals a credential* uses it from outside. Sender binding makes the stolen token useless without its key, and a short lease caps what is left. The detection time does not matter. An attacker who *runs code inside the workload* renews leases as the workload. There the window is set by detection, so Reach and budgets do the work. This is why the architecture never relies on short lifetimes alone.

LIMITS OF THE MODEL

The bound counts what authority allows. It does not count harm from data already read, which cannot be recalled, from denial of service, or from paths the enforcement layer does not cover. An uncovered path has unbounded reach.

02 / SCOPE AND EVIDENCE

Research basis and threat model

This paper is an architectural synthesis of public standards, implementation documentation and research reviewed on 24 October 2026. It presents no prototype measurements. Product documentation establishes supported mechanisms and their stated assumptions. It does not validate a complete deployment. Cited research is used within the scope of the system it studied. MUST and SHOULD express requirements of this reference architecture; they do not amend the cited standards or imply endorsement by their authors.

Attacker capabilities

Threat	Required boundary	Residual exposure
Malicious package or application process	Independent identity issuance and resource enforcement	Permitted business operations and data already reachable
Compromised build job or developer credential	Isolated builders; release approval outside that job	Malicious approved source or a compromised release authority
Compromised maintainer or dependency, validly signed	Source review; build from source; runtime limits	Logic that stays inside its permitted scope
Stolen workload token	Short leases; audience and sender binding	A compromised caller that also holds the binding key
Prompt-injected AI agent	Deterministic tool gateway; task-scoped delegation; no secrets in model context	Actions the user’s delegation allows
Stolen issuer or signing key	Scoped issuers; short issuer leases; transparency logs	Tokens minted within the key’s remaining scope and lifetime
Compromised host or provider administrator	Host-adversarial profile with protected guest enforcement	Hardware or firmware failure, guest compromise, denial of service
Physical access to memory buses	Declared out of scope by current TEE vendors; physical security	Guest secrets and attestation keys on affected platforms
Compromised fabric authority	Scoped issuers; independent recovery and key rotation	Privileges inside that authority’s scope
Network partition or destructive attack	Explicit expiry behaviour; independently recoverable state	Service interruption and transactions already committed

Assumptions every deployment declares

Every deployment records its trusted computing base: hardware and firmware, the software that can change policy, identity issuers, release signers, attestation verifiers and recovery operators. It also names the attack classes it excludes, such as physical attacks or side channels outside the selected hardware model.

Administrative independence needs different compromise paths. Two services on one root-controlled host do not protect each other from that host. Separate accounts can still share one recovery credential. A review traces who controls signing keys, network bypasses and backup deletion.

PART II · ARCHITECTURE

03 / REFERENCE ARCHITECTURE

Control planes and trust boundaries

The architecture gives responsibility to six logical planes (Figure 2). A small installation may run several planes on shared infrastructure, but it must keep the trust boundaries it claims. NIST separates the policy decision from its enforcement, and its cloud-native guidance applies identity-based controls to distributed services. Google's BeyondProd describes the same shift for its own microservices: trust follows verified workload identity and code provenance, not network location. [9–11]

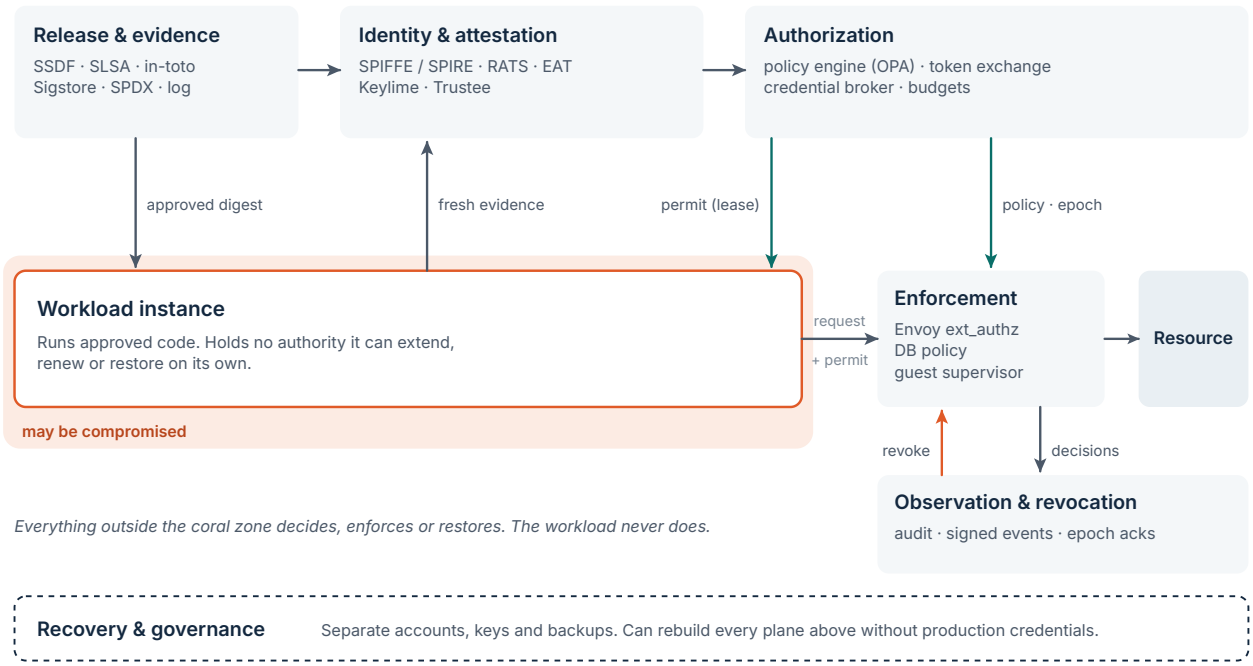


Figure 2. The six planes. Release & evidence supplies reference values to the verifier. The verifier hands an appraised identity to authorization. Authorization issues leases to the workload and policy epochs to enforcement. Observation & revocation acts on enforcement directly. Recovery stands outside production authority.

The figure is a proposed composition. Installing the named components does not by itself make them compatible or give the required properties. An integration defines the identity mapping and the evidence profile, then proves that every sensitive request crosses an effective enforcement point.

Resource-boundary enforcement

The protected resource trusts its gateway or its native authorization mechanism. It never accepts an application's statement that admission succeeded earlier. A backend that can be reached directly is a separate access path. It must apply equivalent checks or be made unreachable.

Independent trust domains

Organizations run local trust domains. Federation names the issuers a domain accepts and the exact resources they may address. Federation is not transitive unless explicitly authorized: trusting a partner does not mean trusting the partner's partners. No universal internet identity provider or global blocking authority is required.

> **ARCHITECTURAL INVARIANT**
>
> The authorization service can reduce or end a workload's authority without that workload's cooperation. The selected profile states whether the host's cooperation may be assumed.

04 / LIFECYCLE

Separate approval, admission and permission

A deployment ties evidence to one running instance. Release approval, admission and per-operation authorization are separate decisions, each made by a separately scoped authority (Figure 3). An approved artifact does not get broad access just because its signature verifies.

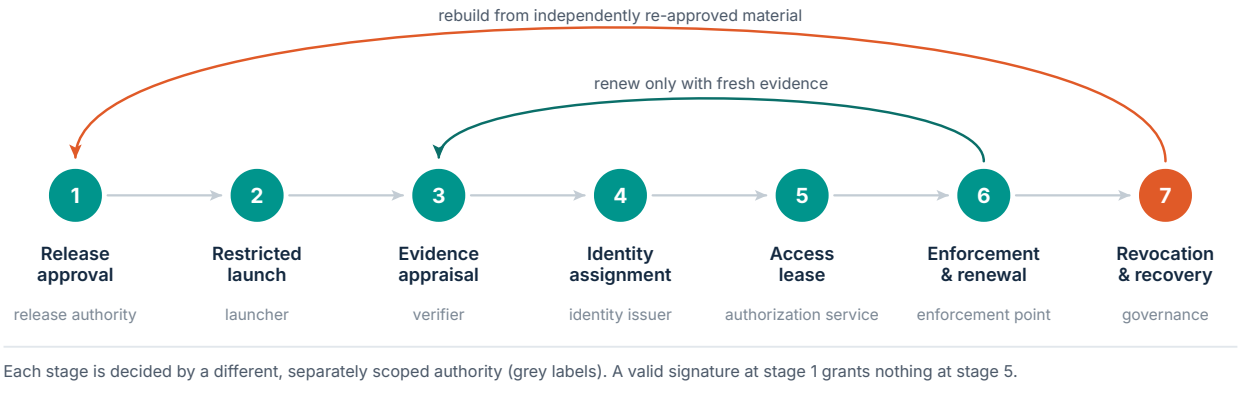


Figure 3. The evidence-bound lifecycle. Renewal returns to evidence appraisal, so authority cannot be extended without fresh evidence. Revocation leads to a rebuild from independently re-approved material, not to a restart of the compromised instance.

Stage	Required decision or binding
1. Release approval	Bind the source revision, immutable artifact digest and configuration to an approved release manifest, recorded in a transparency log.
2. Restricted launch	Start without production secrets. Allow only the bootstrap dependencies needed for verification.
3. Evidence appraisal	Check freshness and the selected execution profile. Bind the evidence to the instance's authenticated channel key.
4. Identity assignment	Map the verified instance to its approved logical workload and tenant. Reject self-selected identity claims.
5. Access lease	Authorize a bounded resource action with an audience, an expiry, a budget and the current policy epoch.
6. Enforcement and renewal	Evaluate each request and end expired sessions. Require acceptable evidence before renewal.
7. Revocation and recovery	Stop issuance, cut existing authority and rebuild from independently re-approved material.

The five design rules, in practice

1. **Prove what runs.** A release manifest names the exact code and every security-relevant setting. A mutable image tag cannot identify an approved executable. Runtime-loaded plugins, mounted code and privileged start-up steps belong to the reviewed launch contract.
2. **Grant per operation.** A workload acts for a specific tenant or authenticated user when the operation needs one. Service identity never supplies missing business authorization.
3. **Make every grant expire.** Every grant has a lifetime and a scope. Long-running sessions get an independently enforced deadline. Delegated authority never outlives the authority that issued it (§15).
4. **Enforce from outside.** A revocation path survives the compromise it is meant to contain. Containing host root needs an off-host resource boundary or an applicable protected execution boundary.
5. **Keep recovery out of reach.** What is needed to rebuild a trusted state stays reachable when production identity and policy services are not.

These are integration requirements. The supply-chain and attestation mechanisms that implement them keep their own trust assumptions. [12, 13]

PART III · MECHANISMS

05 / SOFTWARE SUPPLY CHAIN

Approve the release that actually runs

Protected source and isolated builds

Use SSDF-aligned development practices and target SLSA v1.2 Build Level 3 where the build platform supports it. SLSA's build requirements include protected provenance and build isolation. A project assesses its platform and its own responsibilities before claiming a level; a signature alone does not establish SLSA conformance. [14, 15]

Build workers are separate from production. Ephemeral workers get only the inputs and publishing authority their job needs. They never hold production deployment keys or the power to approve their own output. Untrusted build steps never get a management host's Docker socket. SolarWinds is the reference case: an implant on the build server replaced a source file during compilation, and the vendor's own signed builds carried the result. [1]

Publishing credentials are build secrets too. The Shai-Hulud worm spread by stealing developers' long-lived package-registry tokens and publishing with them. Trusted publishing, in which a CI job proves its identity with a short-lived OIDC token instead of a stored secret, had been generally available for npm since July 2025, six weeks before the worm appeared. [8, 16]

Verifiable release envelope

The release envelope MUST include the artifact digest, the source revision and the builder identity. It also lists the dependency inventory and the security-relevant launch configuration. in-toto can express attestations about each supply-chain step. Cosign can check the expected certificate identity and issuer while it verifies an artifact signature; the release policy pins those expectations to the intended repository and workflow. [13, 17]

Use SPDX for the software bill of materials. An inventory supports vulnerability response and dependency identification. It is not evidence that every component is benign. A vulnerability exception has an owner and an expiry condition. [18]

Record every approval in a transparency log

A signature proves who approved a release. It does not show anyone else that the approval happened. Record each approval in an append-only transparency log, require proof of inclusion at admission, and let independent monitors watch the log for entries the release owner did not expect. Sigstore's Rekor provides this for signatures, Certificate Transparency is the model on the web, and the IETF SCITT architecture generalizes it to supply-chain statements. A stolen signing key then leaves public evidence of its own misuse instead of buying silent trust. [19–21]

Admission binding and update security

The admission controller resolves immutable digests and checks that the approved configuration is the one supplied to the runtime. It rejects unauthorized mounts, changed entry points and unreviewed

executable downloads. Reproducible builds add a consistency check. Matching output does not make the source safe.

Mutable references are an attack surface. In March 2025 the version tags of the tj-actions/changed-files GitHub Action were moved to a malicious commit that printed CI secrets into build logs. Pinning every action and image to a full digest, and giving build jobs only short-lived, narrowly scoped secrets, limits that class of attack. [7]

Use TUF-style signed update metadata with separated roles, threshold signing where appropriate and explicit expiry. Version checks stop an attacker from restoring a release that was valid once and has since been revoked. An emergency rollback is a newly authorized decision with an audit record, not an exemption from rollback protection. [22]

> **MALICIOUS BUT VALIDLY SIGNED**
>
> The xz-utils backdoor came from a co-maintainer identity that had spent more than two years earning trust. The build trigger was only in the release tarballs; the payload was hidden in test files committed to the repository. Every signature check would have passed. Build from reviewed source where possible, compare release archives with the repository, and keep runtime limits and resource authorization mandatory after every supply-chain check succeeds. [4]

Engineering convention

Store the release manifest and its signatures apart from the application image. Production pulls the image by digest and verifies the release authorization independently. Release signing keys are never available to application containers or ordinary build jobs.

---

06 / IDENTITY AND CREDENTIALS

Issue narrow, renewable authority

SPIFFE defines workload identifiers and identity documents. SPIRE issues them using node and workload attestation, including selectors derived from the local platform. A SPIRE identity is not, by itself, hardware evidence that the host is uncompromised. Its assurance depends on the components that assigned it. [23, 24]

Logical identity and instance binding

Give each service role one logical identifier within a trust domain, with explicit tenant and environment boundaries. Record each instance separately. A permit binds the instance to its logical identity, release digest and evidence profile. The application cannot claim another tenant's identity through an environment variable or a request header.

People use phishing-resistant authentication suited to the assurance level; NIST recognizes cryptographic authenticators that resist phishing. Human and workload principals stay separate. Emergency administration needs independent approval and audit. [25]

Credential brokering

Applications ask a broker for an operation or a short-lived credential. The broker validates the workload identity and the delegated user context. Credentials are scoped to an audience and an action. The broker never exposes an unrestricted signing or decryption endpoint.

Where issuer and resource both support it, use certificate-bound OAuth access tokens or DPoP sender-constrained tokens. Both bind the use of a token to possession of a key. They do not help against a compromised caller that holds the key, or a provider endpoint that skips the check. [26, 27]

For an external API that accepts only a long-lived key, keep the key in the broker. The application submits a typed operation. The broker enforces destination and action limits, a tenant scope and a usage ceiling. It must not become a general HTTP relay. A provider credential that can bypass the broker needs its own containment plan.

Delegation that can be audited

When a service acts for a user through another service, exchange the token instead of forwarding it. OAuth 2.0 Token Exchange records the acting party in an actor claim, so the chain of delegation stays visible. Resource indicators restrict a token to one audience. Rich Authorization Requests carry typed, fine-grained permissions such as “export customer records for tenant A”. [28–30]

The issuer is part of the credential

A token is only as narrow as the issuer that signs it. In the Storm-0558 intrusion, a consumer signing key created in 2016 was used to forge tokens that enterprise mail services accepted. The review board found the key should have been retired years earlier. Give each issuer a stated scope and a short validity, and have resources reject tokens from issuers outside that scope. [2]

Authority separation

Identity says who is calling. A lease says which operation is allowed. The application must also establish whose data the operation concerns. Database credentials never turn a verified service identity into access to every tenant.

The public web is moving the same way

Short lifetimes are replacing revocation lists on the public web. CA/Browser Forum ballot SC-081 cuts the maximum lifetime of a publicly trusted TLS certificate from 398 days to 47 days by 2029, and Let’s Encrypt shut down its OCSP service in 2025 and now offers six-day certificates (Figure 4). [31–34]

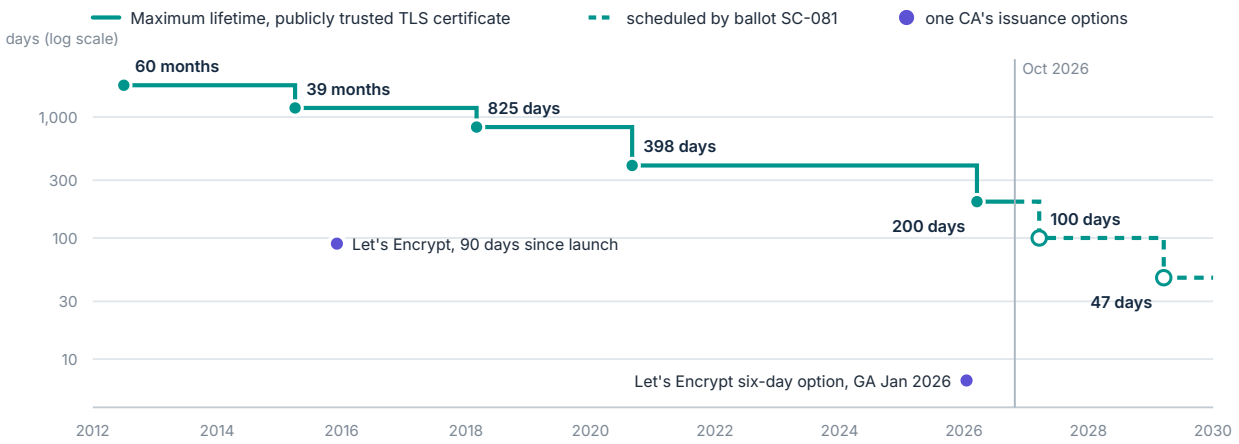


Figure 4. Maximum lifetime of a publicly trusted TLS certificate, by issuance date. Values: 60 months from July 2012; 39 months from April 2015; 825 days from March 2018; 398 days from September 2020; 200 days from 15 March 2026; then 100 days (2027) and 47 days (2029) as scheduled by ballot SC-081. Violet points: Let's Encrypt's 90-day default and its 160-hour certificates, generally available in January 2026.

PROPOSED INITIAL SETTINGS

A pilot may begin with five-minute workload certificates and 60-second leases for sensitive operations. These are engineering starting points, not published best practice or measured safe limits. Test renewals and session deadlines under load and during partitions.

07 / ATTESTATION

Bind evidence to an enforced launch contract

The RATS architecture separates an attester, which produces evidence, from a verifier, which appraises it, and a relying party, which decides. EAT provides a claims format that profiles can adopt. Neither defines a safe workload. The relying party needs explicit claim semantics and trusted reference values. [12, 35]

Freshness and channel binding

A verifier challenges an instance with a fresh nonce. The evidence is tied to the intended audience and to a channel key generated inside the protected boundary when that protection is claimed. The broker checks possession of that key. A report copied from another instance must not release credentials (Figure 5).

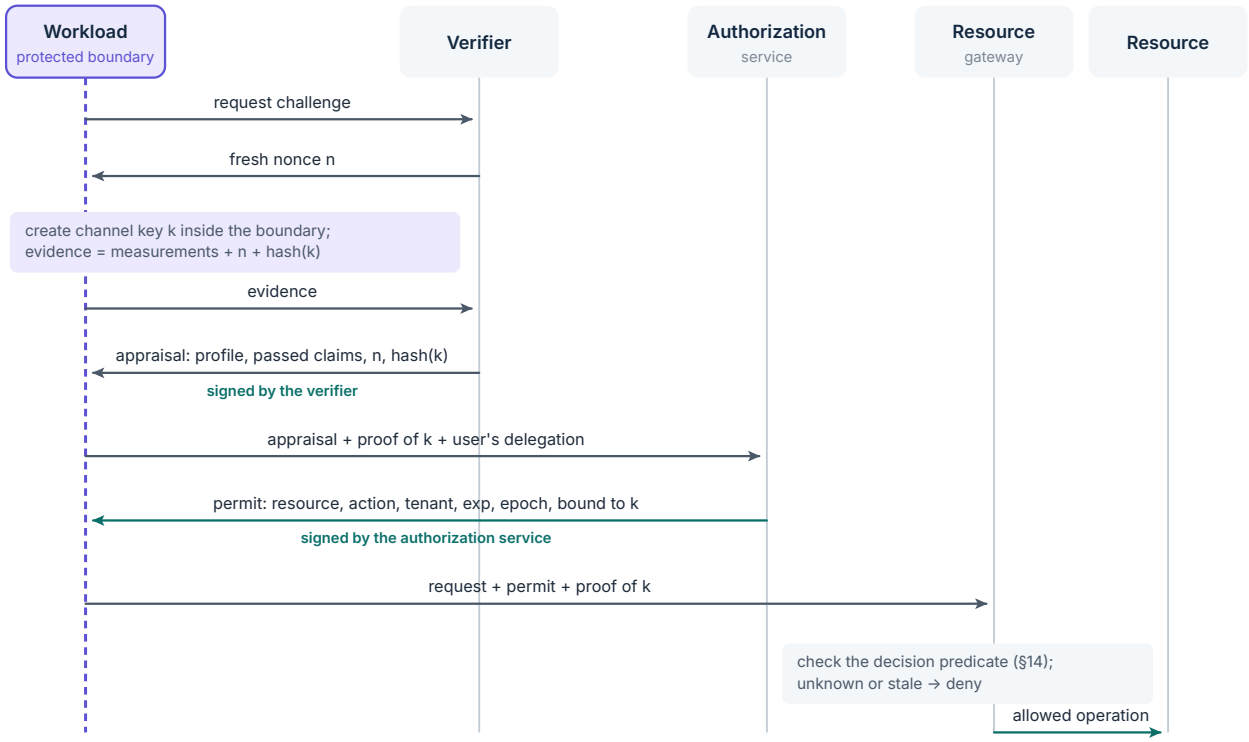


Figure 5. Evidence binding. The channel key *k* is created inside the protected boundary and named in the evidence, the appraisal and the permit, so none of them can be replayed by another instance. Three different authorities sign the release intent, the appraisal and the permit.

A hardware-signed field that carries an application-supplied digest is not enough on its own. A measured, trusted supervisor must enforce the link between that digest and the code actually loaded. Otherwise a malicious host can quote an approved hash while running different software. This follows from the gap between authenticated evidence and the truth of the state it claims.

Two implementation paths

Managed-host path. TPM-backed boot measurements and Linux IMA measurements support integrity appraisal through Keylime. Coverage depends on what is measured and how trustworthy the measurement path is; measured files do not describe every in-memory action. A 2025 case study of Keylime found operational false positives and detection gaps under its test conditions. That motivates explicit coverage tests, not a claim that every deployment shares those defects. [36, 37]

Host-adversarial path. AMD SEV-SNP, Intel TDX and Arm CCA supply evidence for applicable confidential guests. Trustee provides an attestation and key-broker architecture. Confidential Containers documents guest-enforced policy and Init-Data binding that limit what the host may request from a protected guest. The chosen integration must show that the attested supervisor enforces the approved image and configuration. [38–42]

Reference-value governance

Reference values come from an approved release process. They are never accepted because a host happens to be running them. Keep approved firmware and trusted-base versions, with a deliberate

window for patches. Update verification collateral, and retire vulnerable measurements when policy requires.

> **CONTINUOUS EVALUATION HAS LIMITED COVERAGE**
>
> Repeated appraisal detects the changes its evidence represents. It cannot prove continuous benign execution. Publish what each evidence profile observes and which changes remain invisible.

08 / RUNTIME AND DATA

Contain execution and constrain business actions

Application isolation

The managed-host baseline runs services without unneeded privilege: non-root execution, restricted system calls, a mandatory-access-control profile, no writable executable mounts, resource ceilings and no unused capabilities. Docker treats access to its daemon as an administrative capability. Giving an application that access is outside this baseline. [43]

Mounting the Docker socket read-only does not help. The flag protects the socket file, not the API behind it: a process that can connect can still change containers and the host. A service that needs container metadata, such as a reverse proxy doing service discovery, reaches the daemon through a filtering proxy that exposes only the read endpoints it needs. [44]

For an extra isolation boundary, evaluate gVisor or Firecracker against the workload's system-call and operational needs. gVisor runs a sandbox with its own security model. Firecracker provides microVMs. Neither protects a workload from a malicious host administrator. [45, 46]

Host-based eBPF policy and telemetry help within the managed-host profile, and Cilium documents the trust boundaries of its deployment. A compromised host kernel cannot be trusted to enforce rules against itself. The host-adversarial profile therefore places essential decisions off-host or inside a protected guest. [47]

Application and database authorization

A gateway validates the requested action and its tenant context. Use narrow database roles and, where appropriate, row-level security. PostgreSQL superusers and roles with BYPASSRLS bypass row policies, and so do table owners unless FORCE ROW LEVEL SECURITY is set. Application connections never inherit those privileges. [48]

A tenant identifier comes from authenticated delegation and survives every hop without substitution by the caller. A session variable that an attacker can set is not a tenant boundary. Background jobs explicitly own the records they process. Export and deletion need policy that reflects their impact.

Operation budgets

Budgets turn the exposure model's B_r into a control. Each lease carries hard ceilings that the gateway enforces: rows per export, exports per hour, bytes per response, distinct tenants per hour. A budget stops a compromised workload that renews legitimately from draining a store before anyone notices. Set budgets from observed use plus headroom, and alert as use approaches the limit, not only when it is hit.

New software conventions

Use memory-safe languages for new security-sensitive components where the ecosystem allows. CISA and NSA recommend memory-safe development to remove a large class of vulnerabilities. Unsafe interfaces and foreign-function boundaries still need review, and memory safety does not fix authorization flaws or malicious logic. [49]

> CONTAINMENT LIMIT
>
> A compromised application may misuse data it is allowed to read. Budgets and resource-side controls bound that exposure only as far as the business operation itself can be constrained.

09 / AI AGENTS

Treat AI agents as workloads with untrusted control flow

An AI agent is a workload whose next action is chosen by a model reading its inputs. That makes its control flow steerable by data.

Prompt injection — instructions hidden in a web page, an email or a file — is first on the OWASP list of risks for LLM applications. NIST's adversarial machine-learning taxonomy treats indirect prompt injection as its own attack class. Filters and model training lower the risk. They do not remove it. [50, 51]

The answer is the same as for any untrusted workload: decide and enforce outside it. The model proposes. A deterministic gateway decides.

The dangerous combination

An agent becomes dangerous when it has three things at once: access to private data, exposure to untrusted content, and a way to send data out (Figure 6a). Each is often harmless alone. Together they let a hidden instruction read a secret and post it somewhere else. [52] Remove one of the three for a task whenever you can. Where all three are needed, the boundary in Figure 6b is required.

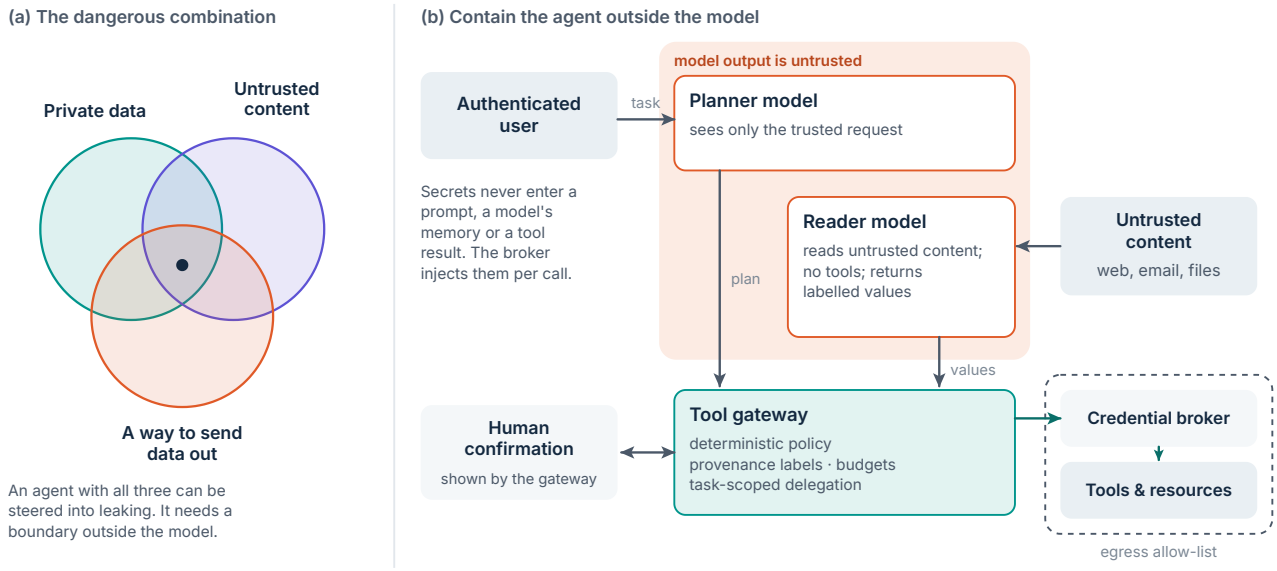


Figure 6. Agent containment. (a) The combination that makes an agent exploitable. (b) The planner sees only the trusted request. Untrusted content goes to a reader that cannot call tools. A deterministic gateway checks every proposed call against policy and provenance before a broker attaches credentials.

Rules for agent authority

1. **No ambient credentials.** Secrets never enter a prompt, a model's context or memory, or a tool's output. Tools call a credential broker that attaches scoped credentials per call.
2. **Authority follows the user's intent.** The agent acts under a task-scoped delegation from the authenticated user, issued through token exchange with the agent named as actor. It is short-lived and lists typed actions. A sub-agent gets narrower authority than its parent, never equal. [28]
3. **Separate planning from untrusted data.** The plan comes from the trusted request. Untrusted content is read by a component that cannot call tools and returns values marked with their source. CaMeL shows the pattern: a privileged model writes the plan, a quarantined model parses untrusted data, and an interpreter tracks where every value came from and enforces policy at each tool call. [53]
4. **Deterministic policy at the gateway.** The gateway sees the action, its arguments and the source of each argument. Typical rules: untrusted data may not choose a recipient, a destination URL or a command; data from one tenant may not flow to another; every outbound send draws on a budget.
5. **Confirm irreversible actions.** Payments, deletions, external messages and permission changes need human confirmation. The gateway, not the model, shows the real parameters.
6. **Egress is part of reach.** Network egress from agent tools is allow-listed. Allowing an entire shared hosting domain is not an exfiltration control.
7. **Instructions are code.** System prompts, tool manifests and MCP server configurations are versioned, reviewed and released like code, with digests in the release envelope. A changed instruction is a new release.

MCP servers are resources

A Model Context Protocol server is a workload with its own identity and its own authorization. The current MCP authorization specification builds on OAuth 2.1. Servers publish protected-resource metadata, clients name the target server with a resource indicator, and a server may not accept or pass through tokens issued for anyone else. Treat each server as a resource behind its own enforcement point, and give it the narrowest tool filter the task allows. [29, 54–56]

> AGENT EXPOSURE
>
> For an agent, Reach is the union of its tool scopes, ρ is its tool-call budget, and W is the task lease. A well-contained agent can still do whatever the user's delegation allows. Containment bounds everything else.

10 / INTERNET INFRASTRUCTURE

Defend transport without a global authority

Routing integrity and source validation

Publish correct route-origin authorizations and deploy RPKI origin validation in stages. NIST provides deployment guidance, and RFC 6811 defines the validation behaviour. Origin validation does not authenticate the full BGP path. A route with no applicable authorization is not the same as an invalid one. [57, 58]

Deploy source-address validation consistent with BCP 38. It stops spoofed traffic. Botnets sending from real addresses remain, so attack absorption and service-level controls are still needed. [59]

DDoS mitigation and service continuity

Use upstream mitigation sized for the threat model. Validate the protected origin path and remove public bypasses to a shielded backend. Critical services need tested DNS recovery and independent administrative access. Check geographic separation against shared upstream and account dependencies.

DOTS defines an open signal channel for requesting DDoS mitigation. A fabric can send authenticated mitigation requests through participating providers if each request covers only resources the requester controls, expires, and is validated by the recipient. This paper does not propose a new universal blocking protocol. [60]

Workload connectivity

Mutual authentication establishes peer identity. Resource policy still decides access. Restrict egress to required services and deny direct paths around the credential broker. A destination allow-list must account for proxy relays and DNS changes.

The network control plane keeps authority over its enforcement devices when an application fails. A compromised host may need an independent switch, a provider firewall or a restricted DPU policy to cut its traffic. Traffic on ungoverned interfaces is outside that claim.

Post-quantum key exchange in transit

Use hybrid post-quantum key exchange on TLS paths that carry data with a long confidentiality life. Chrome has offered X25519MLKEM768 by default since late 2024, and Cloudflare reported in April 2026 that more than 65% of human traffic to its network was post-quantum encrypted. This protects recorded traffic against later decryption. It does not change authentication, which §12 covers. [61, 62]

> AVAILABILITY BOUNDARY
>
> Identity checks cannot restore a saturated link. An operator that controls power or scheduling can still deny service to a confidential guest. Availability needs capacity planning and independently exercised recovery routes.

Privacy convention

Federated decisions disclose only the claims a partner needs. Raw device identifiers and internal measurements are not shared without a purpose. Evidence retention and access policy are part of the trust agreement.

11 / HARDWARE

Make the protected boundary explicit

Hardware is chosen for an adversary model. A processor feature or a security chip does not make all software on the machine safe. The deployment states what is protected and who can change that protection.

Mechanism	Intended role	Boundary that must be stated
Secure or measured boot, TPM	Verify boot policy and authenticate selected measurements	Does not hide guest memory from a host operator
HSM or hardware-backed key	Protect key material against extraction	A compromised but authorized caller can still use the key
SEV-SNP, TDX, Arm CCA	Protect confidential guests under a vendor threat model	Guest code, firmware state and attestation verification stay dependencies; physical attacks are excluded
Confidential GPU	Extend the protected boundary to accelerator memory	Driver, firmware and the GPU attestation chain join the trusted base; GPU evidence must be bound to its guest
Restricted DPU or SmartNIC	Enforce network policy outside host control	Only traffic on the governed path is covered
CHERI capabilities	Enforce capability bounds and compartments in hardware	Needs compatible software and a defined profile

SEV-SNP, TDX and Arm CCA are candidates for the host-adversarial profile, subject to supported platforms and verified launch semantics. The host can still deny scheduling or connectivity. Confidential execution does not make vulnerable guest code trustworthy. [38–40]

Physical access is outside the current threat models

Research in 2024–2026 showed what that exclusion means in practice. BadRAM tampered with a memory module's configuration chip to create memory aliases and broke SEV-SNP attestation; AMD shipped a firmware check. Battering RAM used an interposer costing under $50 on DDR4 buses against Intel SGX and SEV-SNP, and WireTap passively extracted an SGX attestation key. TEE.fail moved to DDR5: it extracted the Intel key needed to forge TDX attestation quotes and recovered signing keys from inside SEV-SNP guests. In September 2026, DDRop showed that dropping DDR5 writes breaks the integrity of SGX, TDX and SEV-SNP, because none of them checks memory freshness. For Battering RAM, WireTap and TEE.fail, the vendors pointed to threat models that exclude physical access. [63–68]

Two lessons follow. A host-adversarial claim must state who can physically touch the machine. And evidence must be bound to its receiver: TEE.fail paired a forged CPU attestation with a genuine GPU attestation relayed from another machine, because the GPU report was not tied to a specific confidential guest. [67, 69]

Network and capability hardware

NVIDIA documents a restricted BlueField mode that separates host privileges from DPU administration. The mode must be enabled and its management credentials isolated; deploying the card does not establish that separation. The DPU's firmware and management environment join the trusted base. [70] CHERI provides hardware capabilities for memory access and compartments. It is an optional evaluation track here, and the baseline does not depend on it. [71]

Firmware is an operational dependency

Keep authenticated firmware updates and independently usable recovery, consistent with NIST platform-firmware resiliency guidance. Hardware-backed claims must follow security advisories. AMD-SB-3019 describes a microcode signature-verification flaw affecting SEV-SNP protections, and its mitigations. A verifier that ignores affected firmware accepts an obsolete assumption. [72, 73]

12 / CRYPTOGRAPHY

Migrate first where trust lives longest

Every boundary in this paper rests on signatures and key exchange. NIST published its first post-quantum standards in August 2024: ML-KEM for key establishment, and ML-DSA and SLH-DSA for signatures. Its draft transition plan proposes deprecating today's quantum-vulnerable algorithms at the 112-bit security level after 2030 and disallowing them after 2035. [74–77]

Finite authority helps here. An object that lives for minutes can switch algorithm at its next renewal. The work concentrates on what lives for years.

Object	Lifetime	Migration priority and approach
Leases, permits, workload certificates	minutes	**Low.** Switch at renewal once issuers and verifiers support the new algorithm.
Key exchange for data that must stay secret	as long as the data	**High now.** Recorded traffic can be decrypted later. Use hybrid ML-KEM key exchange (§10).
Release and update signing keys	years	**High.** Plan ML-DSA or SLH-DSA, or hybrid signatures, with TUF-style key rotation.
Firmware signing and hardware roots	device lifetime	**Highest.** Stateful hash-based signatures (LMS, XMSS) are already specified, and NSA's CNSA 2.0 sets 2030 for exclusive use of quantum-resistant firmware signing in national security systems. [78, 79]
Transparency logs, archived evidence	indefinite	**High.** Use algorithm-agile log formats, or plan to re-sign checkpoints.
Backups encrypted for long retention	years	**High.** Re-wrap data keys under post-quantum or hybrid schemes.

Conventions

Every signed or encrypted object names its algorithm explicitly. Verifiers never infer it from attacker-controlled data. Policy lists accepted algorithms per object type, so a downgrade is a policy violation, not a parsing choice. Inventory where each algorithm is used before choosing a migration order. Test the rotation of every trust anchor now: the hard part of a migration is rotation, not the new mathematics.

> DO NOT WAIT FOR CERTAINTY
>
> No one knows when a quantum computer will be able to break today's public-key cryptography. Long-lived roots must move before that happens, and the path to move them must already have been tested.

PART IV · CONTRACTS

13 / INTEROPERABILITY

Specify what each claim means

A deployable fabric needs a versioned implementation profile. The profile defines the supported evidence and authorization formats, including their failure behaviour. Existing protocols stay unchanged. Profile-specific metadata is clearly namespaced and validated; it never silently reinterprets a standard field.

Convention	Proposed requirement
Identifiers and scope	Stable logical workload ID; separate instance ID; explicit environment and tenant; no wildcard federation by default
Artifact identity	Full content digest with an algorithm identifier; configuration and launch-policy digests where security-relevant; transparency-log entry reference
Evidence semantics	Named profile version, measured boundary, nonce, verification audience and reference-value policy version
Authorization	Resource and action, authenticated delegation, expiry, issuer authority expiry, budget, sender binding where supported, policy epoch
Signed objects	Explicit object type and issuer; approved algorithms and keys per object type; reject unknown critical fields and ambiguous encodings
Revocation events	Unique event ID, scoped subject, durable monotonic epoch, effective time and acknowledgement state
Audit records	Decision ID, subject and release binding, policy version, outcome and expiry; sensitive values excluded

Serialization and signature discipline

Use the serialization and signature rules of the selected envelope. A custom JSON object may use a defined canonicalization profile, such as RFC 8785, where appropriate. The implementation specifies exact bytes and field handling. It never mixes incompatible canonicalization rules or infers the algorithm from attacker-controlled data. [80]

Policy distribution

OPA supports signed bundles and keeps its previous bundle when a new one fails verification. Its documentation notes that some JWT claims, including `iat` and `iss`, are not interpreted by bundle verification itself. A fabric therefore implements and tests its own freshness and issuer-selection rules. Bundle signature verification is configured as mandatory, and a valid signature does not show that a bundle is the currently authorized policy. [81]

A signed, monotonic policy epoch and an explicit maximum validity establish when a cached decision expires. Each enforcement point persists its accepted epoch and resynchronizes after losing state. An administrative rollback is a newly authorized version with its reason recorded.

> **NO UNDEFINED TRUST SCORE**
>
> An attestation result exposes the claims that passed and the profile used to evaluate them. Authorization never relies on an unexplained aggregate score that hides an unmet requirement.

14 / AUTHORIZATION CONTRACT

Separate release intent from runtime permission

The example defines an illustrative release-intent object. It is a schema specific to this proposal, with no out-of-the-box runtime support. Placeholder digests are intentionally invalid. A real implementation needs schema validation and a signed envelope with authorized issuers.

```
type: fabric.example/release-intent
schema_version: "0.2"
subject: "spiffe://example.org/prod/tenant-a/exporter"
release:
  manifest_digest: "sha256:<64-hex-digits>"
  image_digest: "sha256:<64-hex-digits>"
  transparency_log_inclusion_required: true
execution:
  profile: "managed-host-v1"
  launch_policy_digest: "sha256:<64-hex-digits>"
  evidence_max_age_seconds: 300
access:
  audience: "customer-export-gateway"
  resource: "tenant-a/customer-export"
  actions: ["create-approved-export"]
  delegation_required: true
  sender_binding: "dpop"
  lease_max_seconds: 60
  nested_expiry: true              # a permit may not outlive its issuer's lease
  session_deadline_required: true
budgets:
  rows_per_export: 50000
  exports_per_hour: 4
failure:
  new_privileged_access: "deny"
  expired_lease: "terminate"
policy_epoch: 42
```

A release authority signs the intent. A verifier independently returns a signed appraisal result bound to a fresh challenge and the authenticated instance key. An authorization service then issues a separate permit that names the instance, the applicable policy, the expiry and the budget. Each object has its own issuer role, and the application cannot create its own attestation result.

The decision predicate

An enforcement point allows an operation only when every term holds:

$$\begin{aligned}\mathrm{Allow}(op) \iff\ & \mathrm{Auth}(c) \wedge \mathrm{Fresh}(ev, \pi) \wedge \mathrm{Bound}(rel, inst) \\ & \wedge \mathrm{Permits}(\ell, op) \wedge \mathrm{Current}(\ell, e) \wedge \mathrm{Within}(\ell, op) \\ & \wedge \big(\neg\, \mathrm{SenderBound}(\pi) \vee \mathrm{PoP}(c, \ell)\big)\end{aligned}$$

$\mathrm{Auth}(c)$	The caller *c* is authenticated as the instance named in the lease.
$\mathrm{Fresh}(ev, \pi)$	The evidence the lease relied on is still within the maximum age of execution profile π.
$\mathrm{Bound}(rel, inst)$	The approved release and launch policy match the running instance.
$\mathrm{Permits}(\ell, op)$	Lease ℓ names this resource, action and tenant, with a valid user delegation where required.
$\mathrm{Current}(\ell, e)$	The lease and its issuer's authority are unexpired on the enforcement point's clock, and the lease epoch is at least the epoch *e* this point has accepted.
$\mathrm{Within}(\ell, op)$	The operation fits the lease's remaining budget.
$\mathrm{PoP}(c, \ell)$	Where the profile requires sender binding, the caller proves possession of the key bound into the lease.

The managed-host example relies on the host for instance measurement and launch enforcement. Choosing a confidential-guest profile requires an attested guest supervisor and a verified binding to its enforced launch policy. Changing the profile string alone does not create hardware-backed protection.

15 / REVOCATION

Bound access after a revocation decision

Certificate lifetime, token lifetime and the lifetime of an established session are separate quantities. A short-lived certificate does not end an existing TLS session. The fabric MUST put an authorization deadline on sensitive sessions and have a tested way to close them.

Revocation transaction

A scoped revocation commits a new policy epoch and stops new permits for the affected subject. Enforcement points receive the event and acknowledge it. Resource gateways close affected sessions, and an independent network control may quarantine the instance. A path that has not acknowledged stays visible as open until its earlier permits expire.

The target set includes every gateway and resource replica that can authorize the operation. A signed event is not evidence that enforcement happened. The audit record distinguishes the event's creation, its receipt and the effective denial. A compromised issuer needs its trust relationship rotated; revoking one workload identity is not enough.

Conditional containment bound

$$T_{\mathrm{stop}} \;\le\; \max_{p \in \mathcal{P}} \; \min\bigl(D_p + A_p,\;\; H_p + S_p + C_p\bigr)$$

T_{stop} is the time from a committed revocation until every covered path p rejects the affected authority. D_p is event delivery and A_p is application, including session closure. H_p is the *expiry horizon*: how long valid authority can still exist on the path if the push fails. S_p is clock error at the enforcement point, and C_p is

the delay from expiry to closure. An unavailable push path has an unbounded D_p, so the expiry term decides (Figure 7).

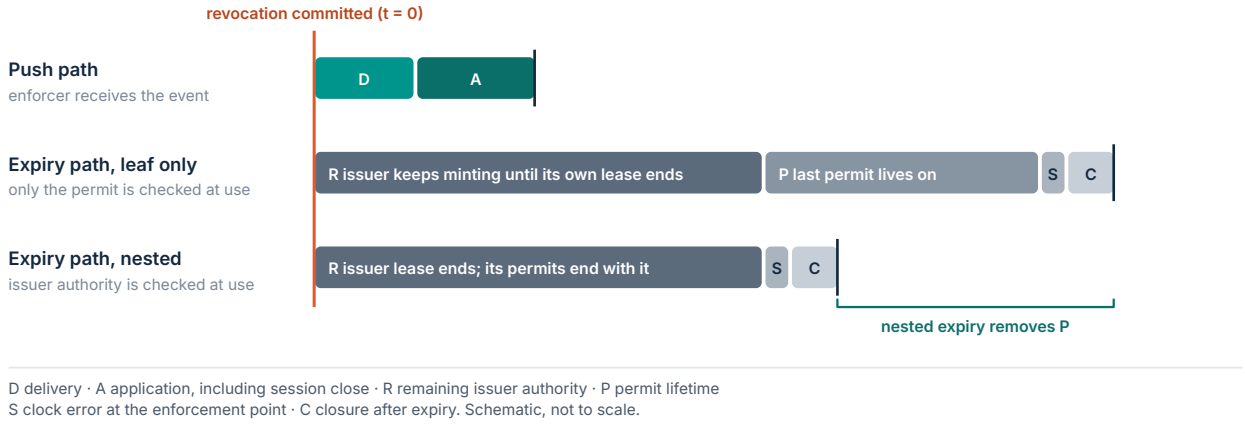


Figure 7. The two ways a path stops honouring revoked authority. Push stops it after delivery and application. Expiry stops it when the last valid permit runs out. Checking the issuer's authority at use (nested expiry) removes the permit lifetime *P* from the expiry path.

The expiry horizon depends on what is checked at use

Let R_p be how long an issuer that never hears the revocation keeps its own authority, and P_p the maximum permit lifetime. If only the permit is checked at use, a cut-off issuer can mint a full-length permit at the last moment. If the enforcement point also checks the issuer's authority at use, as X.509 path validation checks the validity of every certificate in a chain, no permit can outlive its issuer: [82]

$$H_p^{\text{leaf only}} \leq R_p + P_p \qquad H_p^{\text{nested}} \leq R_p$$

The issuer's own clock error drops out of both bounds when the issuer stamps permit expiry with the same clock it uses to check its own lease: if that clock runs late, the issuer mints late but writes an equally early expiry. Clock discipline matters most at enforcement points, which is where S_p sits.

Nested expiry has a practical benefit beyond the bound. Permit lifetime stops mattering for revocation, so permits can be cached longer for performance while issuer leases stay short. The lever that remains is R_p. An issuer whose signing key lives for months offers no useful expiry bound at all (Figure 8).

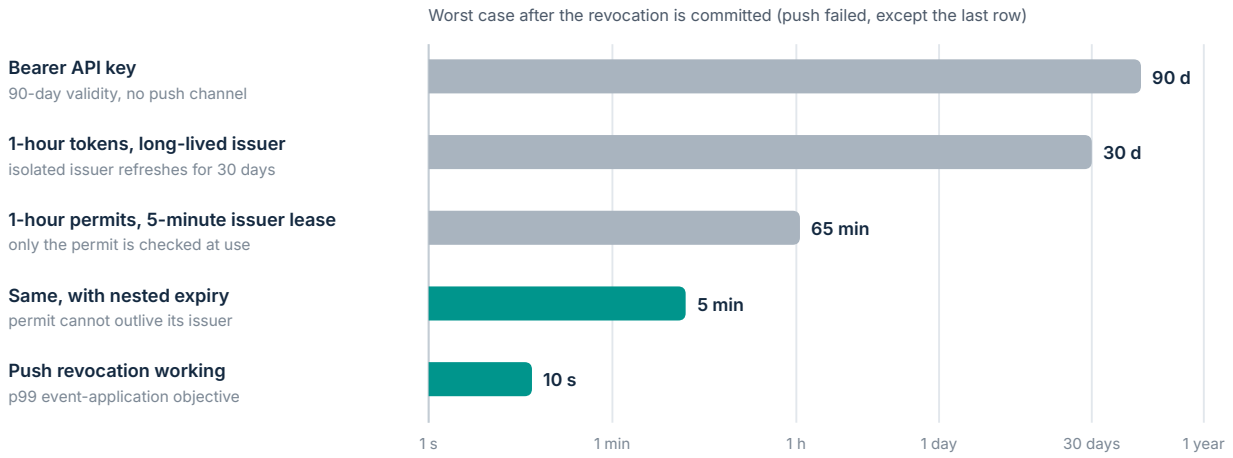


Figure 8. Worst-case time until revoked authority stops working, by design. *Illustrative parameters:* 1 s clock error at enforcement, 5 s session closure, 10 s p99 event application when push works. Short issuer leases bring the fallback bound from weeks to about an hour. Nested expiry brings it to minutes and makes the 1-hour permit lifetime irrelevant.

This is a proposed analytical bound, not an empirical result. It assumes complete path coverage, trustworthy deadline enforcement and a bounded lifetime for every issuer's authority. Detection and decision time come on top. An uncovered backend path or a compromised enforcement boundary invalidates the bound. Data already read and transactions already committed are not reversed.

Illustrative pilot objectives

A pilot can test a 60-second maximum lease for sensitive operations, a five-minute issuer epoch lease, nested expiry at every gateway and a 99th-percentile event-application target below 10 seconds. These are test objectives, not performance claims. Capacity tests include reconnect bursts and simultaneous credential renewal.

> **ACTIVE STREAMS AND TRANSACTIONS**
>
> Define how a revoked long-running export or database operation is cancelled. Disconnecting a client may not undo a transaction the backend has already accepted. Cancellation behaviour and partial-result handling belong in the resource contract.

16 / FAILURE SEMANTICS

Preserve policy during outages

An access system is incomplete until it says how it behaves during partial failure (Figure 9). Availability choices are recorded per resource. Silent fail-open behaviour for privileged operations is prohibited.

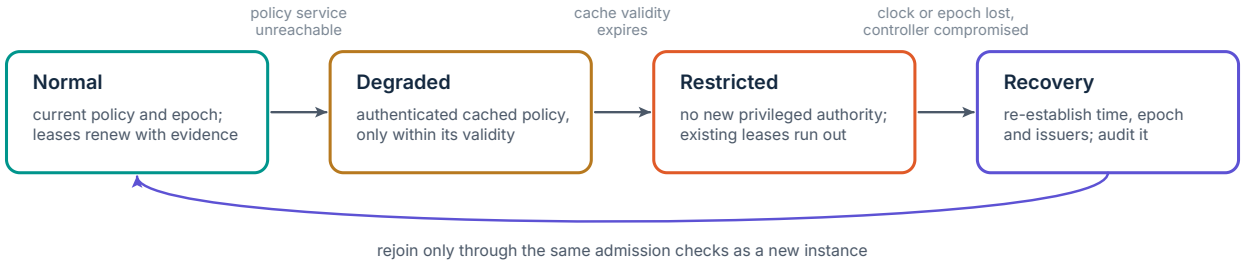


Figure 9. Failure states. Each transition reduces authority or holds it at its existing deadline; none extends it. Recovery returns to normal only through the admission checks a new instance would face.

Failure condition	Required behaviour
Identity issuer or attestation verifier unavailable	Issue no new elevated authority. Existing bounded permissions continue only until their previously authorized deadline.
Policy service unreachable	Use authenticated cached policy only within its approved validity. Deny new privileged access when that validity expires.
Policy signature fails, or epoch decreases	Reject the update. Keep the last acceptable policy only while it remains valid, and raise an operator-visible incident.
Revocation channel partitioned	Expire permissions locally and end covered sessions. Never silently extend authorization to preserve service.
Clock integrity or persisted epoch lost	Enter a restricted recovery state. Re-establish trusted time and policy state before granting new sensitive authority.
Audit destination unavailable	Use protected, bounded buffering. Deny operations whose policy requires durable audit before execution.
Enforcement controller compromised	Revoke its authority from an independent boundary and start the clean recovery process.

Local decisions and bounded caching

Policy evaluation may run near a protected resource to avoid a synchronous global dependency. The local evaluator authenticates its configuration and respects lease deadlines. Envoy's external-authorization filter is an enforcement integration point; deployments set its failure behaviour explicitly and test every protected route. Shadow-mode evaluation is not an enforcement guarantee. [83]

The control plane has a blast radius too

On 19 July 2024 a faulty content update for the CrowdStrike Falcon sensor reached millions of Windows hosts at once and crashed them. The update channel was trusted, privileged and unstaged; the vendor has since committed to canary and ring-based deployment of that content. Apply this paper's rules to the fabric itself. Stage policy and evidence-profile rollouts with canaries, scope automated containment by service or tenant, and keep an independent rollback path. A single noisy detector never gets the power to disable the whole fleet. [5]

Bootstrap without circular dependencies

Keep a restricted path to the trust anchors and verification services needed for enrolment. Recovery never depends only on the identity issuer being recovered. Document the independent mechanism that restores that issuer, including protected administrative credentials and the network path used during an outage. Emergency overrides need a named operator and a bounded validity, and they cannot silently remove the audit trail.

17 / GOVERNANCE AND RECOVERY

Recover the authority to trust a system

Separation of administrative powers

The release owner approves application intent. The security authority approves evidence profiles and resource policy. Operations staff carry out approved deployments. Small teams may hold several roles, but high-impact root changes need an independent approval path, and separate signing keys express the roles even when people overlap. Changes to trust roots require a quorum (for example two of three key holders) and are recorded in the same transparency log as releases.

Every authority has a documented scope and a revocation procedure. Root and recovery keys stay outside ordinary production workloads. Hardware-backed keys help where supported, but the operations they allow still need authorization. Management-plane access is restricted, and emergency use is recorded in an independent audit destination.

Recoverable evidence and state

CISA recommends protected backups and tested restoration as ransomware preparation. The fabric keeps independently accessible backups of essential application data and of the configuration needed to rebuild trust services. A backup that production administrators can erase with the same credential does not meet the independence requirement. [84]

Object-lock mechanisms preserve retained versions, and S3 documents distinct governance and compliance retention modes. Retaining bytes does not show that the data is clean, that every dependency was backed up, or that the decryption key still exists. Protect retention settings and key lifecycle independently. [85]

Recovery sequence

Establish a clean recovery control plane with reviewed trust anchors. Suspend the compromised issuing authority and preserve incident evidence. Reassess the approved release set, including the builder and the signer. Deploy a clean instance and reissue only the permissions it needs. Restore a selected data checkpoint after checking application consistency and signs of contamination.

Rejoining production requires the same admission checks as a new instance. A recovery deployment is never exempt from policy because the service is urgent. A separately approved emergency profile may reduce functionality while keeping the minimum containment boundary.

RESTORE TEST AS A SECURITY TEST

An acceptance exercise rebuilds from a state in which the production identity service and the ordinary administrator account are unavailable. It verifies application correctness and key availability, not merely that files came back.

Operational ownership

Assign an owner to each evidence profile and each trust root. Record supported hardware and software versions. Run an advisory-response process for changes to that trusted base. Publish the maximum accepted age of verification collateral and the exception process for a failed patch rollout.

PART V · OPERATIONS

18 / DEPLOYMENT PROFILES

Declare the protection actually deployed

A profile names who the deployment trusts and who it assumes may be hostile (Figure 10). The two profile names are proposed conventions. Each comes with a versioned conformance statement that lists the actual controls and exclusions. Hardware attestation is never a cosmetic badge on an otherwise unchanged managed-host deployment.

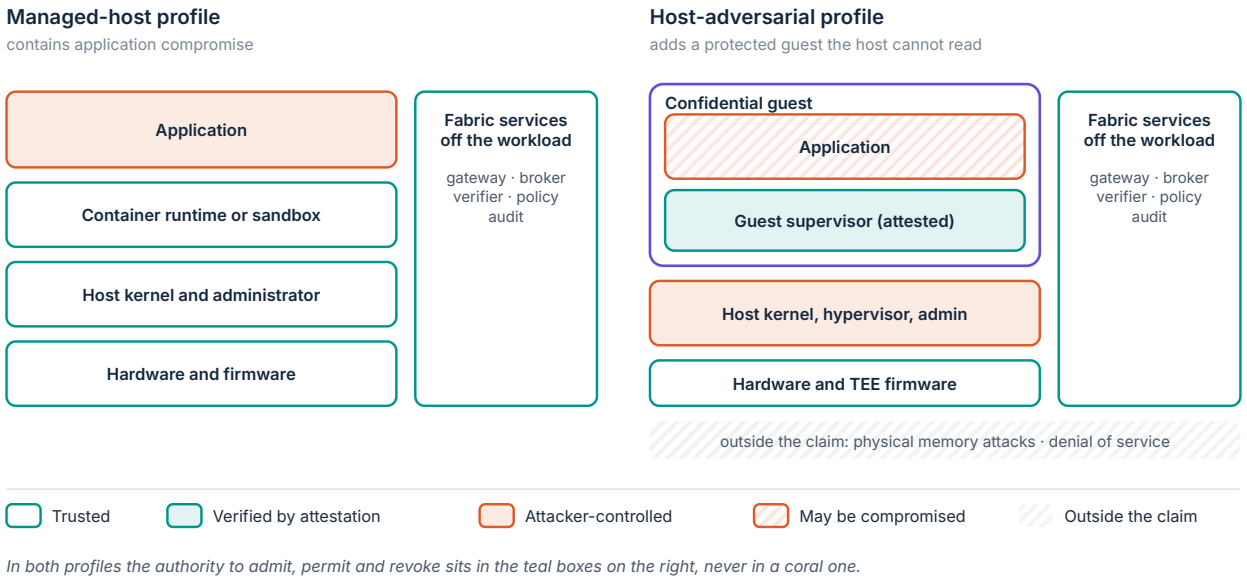


Figure 10. Who is trusted in each profile. The managed-host profile trusts the host and contains the application. The host-adversarial profile treats the host as hostile and relies on a confidential guest whose supervisor is verified by attestation; the application inside it may still be compromised.

Property	Managed-host profile	Host-adversarial profile
Trusted host	Host administrator, kernel and hypervisor are trusted	Host operator is outside the selected confidentiality and integrity assumptions
Execution boundary	Hardened container, or a suitable sandbox or microVM	Supported confidential guest with a trusted guest supervisor
Instance evidence	Trusted platform selectors; optional TPM and IMA appraisal	Verified TEE evidence bound to the enforced guest launch policy
Credential release	Broker validates identity and the approved release	Off-host broker also validates the confidential execution profile
Resource enforcement	Off-workload gateways and native resource authorization	Off-host enforcement or applicable protected-guest enforcement
Host compromise claim	No resistance to host root is claimed	Only properties covered by the selected hardware and guest threat model
Physical access	Covered by facility security only	Declared: current TEEs exclude memory-bus interposers (§11)
Availability	Independent redundancy and recovery required	Independent redundancy and recovery required

Control placement determines the claim

A policy proxy beside the application can contain a constrained application process if the isolation boundary holds. Host root may control both. A remote database gateway can still deny that host access if its credentials and administration are independently protected. A confidential guest may protect internal keys from its host, subject to the hardware model and the correctness of the guest components. [38, 39, 43]

Integration surface

SPIRE, OPA and a gateway provide major building blocks, but the full admission contract needs integration work specific to each implementation. Trustee and Confidential Containers provide an attestation path for supported environments. The instance permits, policy epochs, nested expiry and end-to-end revocation guarantees in this paper must still be implemented, or mapped to equivalent supported controls, and tested. [24, 41, 42, 81]

> CONFORMANCE DISCLOSURE
>
> A deployment identifies which controls are enforced, which are observed only and which are absent. No broad “verified” label may hide a missing workload binding or an unprotected direct path to a resource.

19 / PRACTICAL REFERENCE DEPLOYMENT

A Docker and Coolify adoption path

The managed-host profile can guide an incremental Docker-based deployment without a move to Kubernetes. The table places each boundary; Figure 11 shows the topology. This section is a proposed

operating pattern. It does not claim that stock Coolify implements the full contract, or that any particular live environment has been inspected.

Boundary	Proposed placement and responsibility
Build environment	A separate host for builds. Ephemeral workers produce a signed release envelope and hold no production secrets. Images ship to production by digest; production never builds.
Deployment authority	A restricted management service verifies release approval and log inclusion, then deploys exact digests.
Application hosts	Supported Docker configuration; non-root, read-only workloads; no application-held daemon authority.
Reverse proxy	Reads container metadata through a filtering socket proxy, never the raw daemon socket.
Identity and policy services	A protected service boundary; reviewed SPIRE registrations and signed, versioned policy.
Sensitive resource gateway	An independent host or account where needed. Validates every permit and blocks backend bypass.
Audit and recovery	Independent destination and credentials; immutable retention where appropriate; exercised restore.

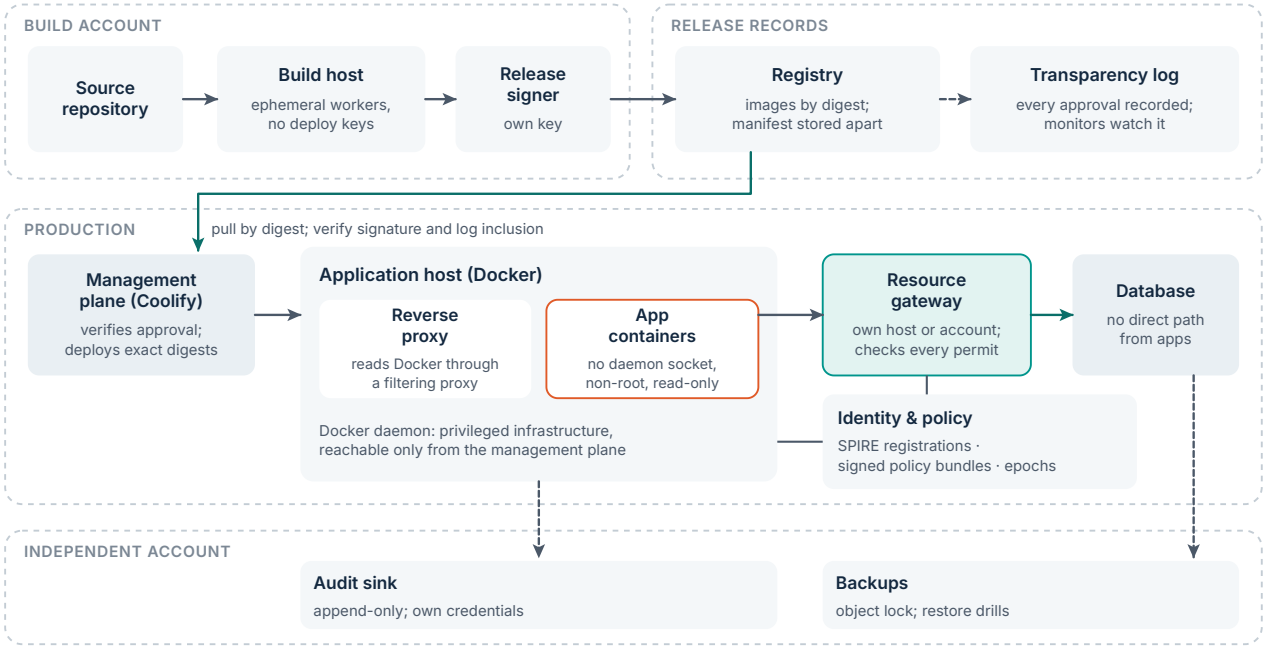


Figure 11. Reference topology for a small Docker estate. Builds run in their own account, approvals land in a transparency log, the management plane deploys only exact digests, and the resource gateway, audit sink and backups sit outside the application host's authority. Dashed arrows carry records, not authority.

Preserve supported operations

Use supported deployment interfaces and external verification hooks. Any extra adapter needs tests and an upgrade-compatibility policy. Never present an unsupported modification of a management product as an established security feature. Where a required enforcement hook does not exist, document the gap and choose a supported enforcement point.

Management-plane exception

A deployment manager may legitimately need broad Docker control. Treat it as privileged infrastructure with a restricted administrative path, not as an ordinary application container. The goal is to stop untrusted jobs and applications from inheriting that authority. Docker's daemon security model makes this separation consequential. [43]

First protected business operation

Choose one sensitive operation, such as exporting one tenant's customer records. Put the export behind an independent gateway. Bind each request to a verified service and an authenticated tenant delegation. Deny the exporter direct database access except where its role really needs it. Apply a 60-second lease, nested expiry, an export budget, and test forced cancellation.

Expand only after the isolated operation passes its revocation and recovery exercises. Adding hardware attestation later needs a compatible execution environment and a new evidence profile. It is never a configuration switch that secures an arbitrary existing VPS.

---

20 / ADOPTION AND ECONOMICS

Advance through evidence-based gates

Roll out by demonstrated properties, not by calendar (Figure 12). Each stage publishes its residual risk and the evidence needed to proceed. A deadline does not establish assurance.

Operating-cost model

The main costs are engineering ownership of policy and evidence profiles, and the availability burden of the identity and verification services. Other drivers are signature and attestation throughput, certificate-renewal load, audit retention and duplicate recovery capacity. Hardware procurement matters only for profiles that require it.

An evaluation measures CPU and memory overhead alongside application throughput. It reports request latency at the median and in the tail, and includes cold start, burst enrolment and mass revocation. A single microbenchmark is never extrapolated into a claim about fleet-wide security or performance.

Avoiding a control-plane bottleneck

Replicate issuance and verification services where the security model allows. Keep resource enforcement able to apply a valid local policy without a synchronous central call on every operation. Local caching keeps the expiry and anti-rollback rules, and nested expiry lets permits be cached longer without lengthening the revocation bound. A resilient implementation still denies privileged access once its authorized cache validity ends.

Procurement convention

Ask each provider or component owner to state the claim semantics it supports and the failure modes it has tested. "Supports attestation" is not enough without a documented binding to the application that

receives the secret. “Signed images” is not enough without approved signer selection and verification of the image actually launched.

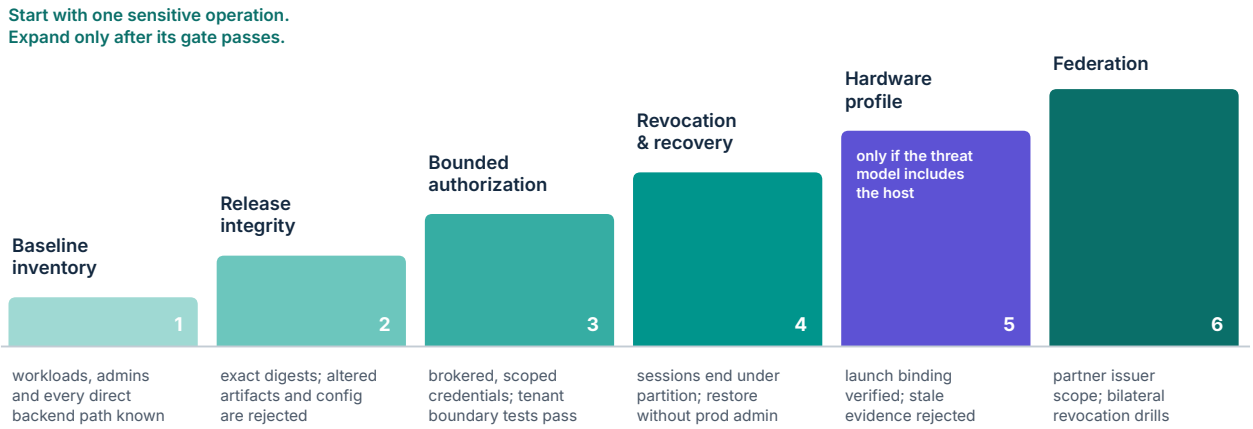


Figure 12. Adoption gates and their exit evidence. Stages 1–4 apply to every deployment. Stage 5 applies only where the documented threat model requires protection from the host. Stage 6 adds partners.

INVESTMENT PRIORITY

Fund release verification, enforceable access leases and operation budgets first. Adopt the host-adversarial profile when a documented threat model requires protection from host administration and the team can operate its extra trust dependencies.

21 / VALIDATION

Acceptance tests must try to break the claim

The architecture requires negative tests and fault injection. Each result records the component versions and the selected profile. A passed test is evidence for the tested condition, not proof that every attack path has been exhausted. Tests marked **new** were added in version 2.0.

Test	Expected acceptance evidence
Approved signer, wrong repository or builder	Admission denies the artifact and names the failed policy condition
New: valid signature, no transparency-log entry	Admission denies; the monitor reports the attempt
Valid image, modified launch configuration	Unauthorized entry point or executable mount is rejected
Replayed attestation or substituted instance key	Broker refuses release; challenge and channel binding are verified
Stale firmware evidence or retired measurement	Verifier rejects new elevated access under the current profile
Tenant substitution and overbroad data role	Resource denies cross-tenant reads and privileged database bypass

Test	Expected acceptance evidence
Stolen sender-constrained token used with another key	Resource rejects use without the required proof of possession
Direct backend connection or egress relay	Bypass is blocked or independently subject to equivalent authorization
New: permit whose expiry is later than its issuer's lease	Enforcement rejects it, or honours it only until the issuer deadline
New: renewing workload exceeds its lease budget	Gateway denies further operations and raises an alert
Revocation during a long-running export	Covered streams close within the declared bound; partial outputs are recorded
Partitioned revocation or policy service	Permissions expire locally; no unapproved lease extension occurs
Old signed policy after a restart	Durable epoch prevents rollback; unsynchronized instances cannot elevate access
New: faulty policy bundle in a staged rollout	The canary fails, rollout halts, and the previous policy stays valid
New: retrieved content tells an agent to send data to a new recipient	Gateway blocks it: untrusted data may not choose a recipient
New: sub-agent requests a wider scope than its parent	Token exchange refuses the request
New: agent context, memory and logs scanned after a task	No secrets present; the broker log shows every credential use
New: signature algorithm downgraded in a signed object	Verifier rejects it as a policy violation
New: MCP server receives a token issued for another audience	Server rejects it and does not pass it upstream
Host-root control in the managed-host profile	Report explicitly marks host-root resistance as outside the claim
Host interference in the confidential profile	Selected protected secrets stay unavailable in tested cases; denial of service remains possible
Compromised issuer or inaccessible production account	Independent root rotation and restore complete and are audited

Execution discipline

Run destructive tests in an authorized environment with representative data and recovery capacity. Record unsuccessful attack attempts and any successful bypass. Repeat the relevant tests after changes to the runtime, firmware, model or trusted policy. A detector alert without effective denial does not pass an enforcement test.

22 / MEASUREMENT AND LIMITATIONS

Measure bounded harm and operational correctness

Metrics with security meaning

Report each term of the exposure model, so a reader can see where harm is still unbounded.

Term	Metrics
Reach	Sensitive operations with complete enforcement coverage; open bypasses; the most data reachable under one workload's authorized role
Rate and budget	Budgets set against observed peaks; budget denials; time to drain a store at the permitted rate
Window	Time to detect and time to decide, reported separately from T_{stop}; the slowest tested covered path and the tail of the distribution
Correctness	False-denial rate; expiry enforcement during outages; recovery-point loss; time to restore trusted service

A count of installed products does not establish any of these quantities.

Known assurance gaps

Attestation coverage is bounded by the evidence collected. The Keylime case study shows how hard it is to evaluate runtime integrity in changing systems. Heterogeneous TEEs and runtime stacks need explicit interpretation of measurements; a common token format does not make every vendor's evidence equivalent. [35, 37]

An authorized malicious release can operate inside its permitted scope. Colluding administrators or compromised roots may authorize harmful policy. A confidential guest can contain vulnerable code, and physical attacks remain outside current TEE models. A model inside an agent can be steered within whatever its delegation allows. Network partitions make immediate, globally consistent revocation impossible without cutting off isolated paths; the proposal resolves that trade-off with finite local deadlines for sensitive operations.

Research and engineering agenda

- **Workload binding.** Independently reviewable tests connecting protected instance keys to the exact executable and launch policy, including late-loaded code and state migration.
- **Revocation semantics.** Validate maximum session duration across proxies and databases. Implement nested expiry in common token formats and gateways. Explore epoch recovery after state loss without allowing rollback or permanent denial.
- **Policy correctness.** Test whether generated least-privilege policies keep intended business operations while excluding prohibited ones. Policies produced by AI require deterministic validation and accountable approval.
- **Agent containment.** Measure how much useful work provenance-tracking gateways allow on real tasks, and publish the policies that make the trade-off acceptable.

- **Cryptographic migration.** Exercise post-quantum and hybrid rotation of release, firmware and transparency-log keys end to end.
- **Interoperability.** Publish a minimal profile specification with reference test vectors. Each implementation discloses unsupported claims and any downgrade behaviour.

23 / REQUIREMENT TRACEABILITY

Map every promise to an observable control

The identifiers below are proposed conformance requirements. The cited foundations establish relevant mechanisms; the integrated guarantees need the acceptance evidence described here. Requirements R-13 to R-17 are new in version 2.0.

ID / requirement	Implementation evidence	Foundation
R-01 / Declare the trusted boundary	Profile names host assumptions and the complete trusted computing base	NIST zero trust; RATS [9, 12]
R-02 / Bind approval to execution	Approved artifact and launch configuration match the running instance	SLSA; in-toto; Cosign [13, 15, 17]
R-03 / Authenticate workload identity	Registration and instance mapping reject caller-controlled impersonation	SPIFFE / SPIRE [23, 24]
R-04 / Bind evidence to the receiver	Fresh attestation and channel key are tied to the enforced launch state	RATS; Trustee; guest policy [12, 41, 42]
R-05 / Constrain each operation	Resource checks audience, tenant and action, with no ungoverned bypass	NIST cloud-native access; Envoy [10, 83]
R-06 / Limit credential use	Lease scope and supported sender binding are enforced by the resource	OAuth mTLS; DPoP [26, 27]
R-07 / Enforce policy freshness	Signed version, validity and durable epoch reject rollback	TUF; OPA [22, 81]
R-08 / Terminate revoked authority	All covered paths enforce the tested lease and session deadline	Proposed integration requirement
R-09 / State outage behaviour	Partition tests demonstrate explicit denial and recovery behaviour	Proposed integration requirement
R-10 / Protect and recover roots	Independent recovery restores service after loss of production authority	Firmware resilience; recovery guidance [72, 84]
R-11 / Maintain the hardware profile	Firmware and collateral policy excludes unacceptable trusted-base states; physical-access exclusions are declared	Vendor attestation and advisories [38, 39, 73]
R-12 / Publish limitations	Conformance report lists residual risks and unimplemented controls	Proposed governance requirement
R-13 / Log every approval	Admission requires proof of log inclusion; monitors watch for unexpected entries	Rekor; CT 2.0; SCITT [19–21]

ID / requirement	Implementation evidence	Foundation
R-14 / No authority outlives its issuer	Enforcement checks issuer authority at use; permits beyond it are rejected	X.509 path validation [82]
R-15 / Budget every sensitive lease	Gateway enforces per-lease ceilings and alerts as use approaches them	Proposed integration requirement
R-16 / Contain agent authority	No secrets in model context; task-scoped delegation; deterministic gateway; human confirmation for irreversible actions	OWASP LLM; CaMeL; MCP; token exchange [28, 50, 53, 54]
R-17 / Plan cryptographic migration	Algorithm inventory; explicit algorithm IDs; tested rotation plan for long-lived roots	FIPS 203–205; SP 800-208 [74–76, 78]

A conformance report includes the release set, profile versions and test results. It names the parties authorized to change each trust anchor. Claims are limited to the tested configuration and the named threat model. Certification or independent evaluation would need a separate process.

24 / CONCLUSION

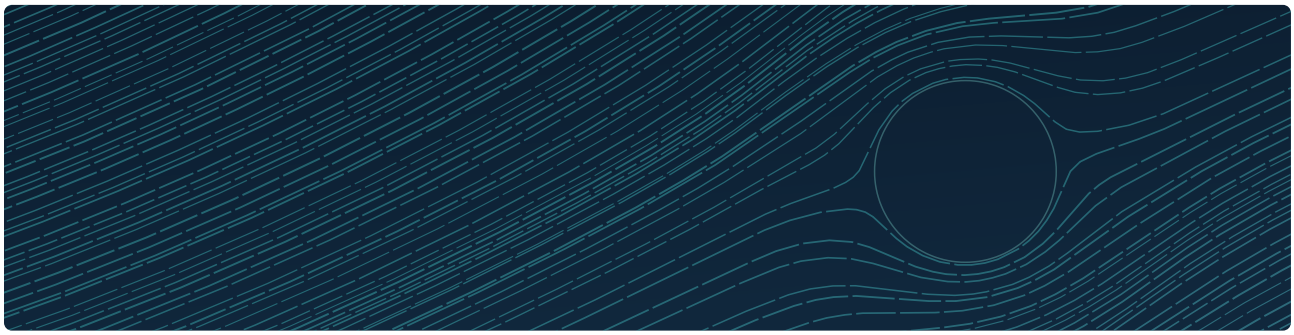
A tested limit on authority after compromise

Verified Compute Fabric is credible only when every access claim traces to authenticated evidence, an accountable policy decision and an enforceable boundary.

Its deliverable is not a product or a score. It is a tested limit on the authority that survives a compromise, written as numbers someone else can check: what was reachable, how fast it could be used and how long it lasted.

Version 2.0 adds three things: a way to compute the limit; a rule, nested expiry, that keeps the limit short when revocation messages fail; and coverage of AI agents, the first workloads whose control flow can be steered by the data they read.

This paper proposes the contract and the validation program needed to establish that limit. It does not claim the limit has been demonstrated. The next step is to build the first protected operation and measure it.



Bounded Weave, seed 1024: the fabric after recovery. The boundary remains as a faint ring.

APPENDIX A / GLOSSARY

Terms used in this paper

Attestation
A process in which an attester produces signed evidence about its state and a verifier appraises it against reference values.

Budget
A hard ceiling on what a lease may do, such as rows exported or calls made, enforced by the gateway regardless of how often the lease is renewed.

Confidential guest
A virtual machine or container whose memory is protected from its host by processor features such as SEV-SNP, TDX or Arm CCA, under the vendor's threat model.

Covered path
A route by which an operation can reach a resource and on which enforcement is proven to apply. Any other route is a bypass.

Delegation
Authority to act for a specific user or tenant, carried as a token that names both the acting workload and the party it acts for.

Enforcement point
The component that evaluates each request and allows or denies it: a gateway, a database policy, a guest supervisor or a network control.

Epoch
A signed, monotonically increasing policy version. Enforcement points persist the highest epoch they have accepted and reject anything older.

Evidence profile
A named, versioned statement of what evidence is collected, what it covers and how it is appraised.

Expiry horizon
How long valid authority can still exist on a path if a revocation message never arrives.

Exposure
The harm a compromised authority can do before it stops: bounded by reach, rate, budget and window.

Host-adversarial profile
A deployment profile that treats the host operator as hostile and relies on a protected guest and off-host verification.

Issuer lease
The limited time for which an issuer may mint permits. An issuer cut off from revocation messages stops when its lease ends.

Lease (permit)
A short-lived grant that names a resource, an action, a tenant, an audience, an expiry, a budget and a policy epoch.

Managed-host profile
A deployment profile that trusts the host administrator and kernel and contains application compromise.

Nested expiry
The rule that a permit is valid only while the authority that issued it is valid, checked at the time of use.

Prompt injection
Instructions hidden in content an AI model reads, which steer the model toward actions its user did not request.

Provenance label
Metadata that travels with a value and records where it came from, so policy can treat untrusted values differently.

Reference value
An approved measurement, such as a digest of firmware or code, against which evidence is compared.

Relying party
The component that uses an appraisal result to make a decision, such as releasing a credential.

Sender-constrained token
A token bound to a key (through mutual TLS or DPoP) so that only the holder of that key can use it.

Transparency log
An append-only, publicly auditable log. Inclusion proofs show an entry exists; monitors watch for entries nobody expected.

Trusted computing base
Everything whose failure could break a stated security property: hardware, firmware, kernels, issuers, signers, verifiers and recovery operators.

APPENDIX B / REVISION RECORD

Changes from version 1.0

Version 1.0 (24 October 2026) was drafted with ChatGPT. Version 2.0 keeps its thesis, its two profiles, its failure semantics, its acceptance tests and all 40 of its references. The changes below are additions or corrections.

Area	Change
Structure	Regrouped into five parts. Twelve figures and two pieces of generative artwork added. Caveats that v1.0 repeated in most paragraphs are stated once, in the front matter, and the prose is shorter and plainer.
§1 Problem	New. Seven verified public incidents, each mapped to the boundary it shows. New exposure model, with the distinction between a stolen credential and a controlled workload.
§2 Threat model	Four new threat rows: validly signed malicious dependency, prompt-injected agent, stolen issuer key, physical memory access.
§5 Supply chain	Transparency logging of approvals. Publishing without long-lived tokens. SolarWinds, xz-utils, tj-actions and Shai-Hulud as worked lessons. SLSA reference title corrected.
§6 Identity	Token exchange, resource indicators and rich authorization requests for auditable delegation. Issuer scope as part of the credential (Storm-0558). Certificate-lifetime trend chart.
§7 Attestation	Evidence-binding sequence figure. Arm CCA added.
§8 Runtime	Operation budgets as a control. Read-only Docker socket mounts called out as unsafe.
§9 AI agents	New section. Agents as workloads with untrusted control flow; seven rules; MCP authorization.
§10 §11 §12	Hybrid post-quantum key exchange in transit. Memory-interposer attacks from 2024–2026 and GPU attestation binding. New section on cryptographic agility and post-quantum migration.
§14 §15	Release intent gains log inclusion, sender binding, nested expiry and budgets. The decision predicate is written out term by term. The revocation bound's expiry horizon is split by validation model; nested expiry removes permit lifetime from the worst case; issuer clock error is shown to cancel.
§16 §17	Failure-state figure. The control plane's own blast radius (CrowdStrike). Quorum approval and logging of trust-root changes.
§18 – §23	Profile, topology and adoption figures. Physical-access row in the profile table. Nine new acceptance tests. Metrics tied to the exposure model. Requirements R-13 to R-17.
References	All 40 original references kept and re-checked; exact dates added for NIST SP 800-63B-4 (July 2025) and the CISA/NSA memory-safety sheet (24 June 2025). New references cover incidents, transparency, delegation, post-quantum standards, TEE attacks and AI-agent security, all checked against primary sources on 24 October 2026.

PRIMARY SOURCES

References

Sources consulted 24 October 2026. Titles link to the originating publisher. Maintained documentation changes; implementation reviews should record the exact revision used. References are numbered in order of first citation.

[1] **CrowdStrike Intelligence.** SUNSPOT: An Implant in the Build Process. 11 January 2021.

[2] **Cyber Safety Review Board (DHS).** Review of the Summer 2023 Microsoft Exchange Online Intrusion. 20 March 2024.

[3] **Mandiant (Google Cloud).** UNC5537 Targets Snowflake Customer Instances for Data Theft and Extortion. 10 June 2024.

[4] **A. Freund.** backdoor in upstream xz/liblzma leading to ssh server compromise. oss-security mailing list, 29 March 2024. CVE-2024-3094.

[5] **CrowdStrike.** External Technical Root Cause Analysis — Channel File 291. 6 August 2024.

[6] **D. Weston (Microsoft).** Helping our customers through the CrowdStrike outage. Official Microsoft Blog, 20 July 2024.

[7] **GitHub.** tj-actions/changed-files: compromised version tags (CVE-2025-30066). Security advisory GHSA-mrrh-fwg8-r2c3, 15 March 2025.

[8] **CISA.** Widespread Supply Chain Compromise Impacting npm Ecosystem. Alert, 23 September 2025.

[9] **NIST.** Zero Trust Architecture. SP 800-207, August 2020.

[10] **NIST.** A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments. SP 800-207A, September 2023.

[11] **Google Cloud.** BeyondProd: A new approach to cloud-native security. Whitepaper, December 2019.

[12] **IETF (H. Birkholz et al.).** Remote ATtestation procedureS (RATS) Architecture. RFC 9334, January 2023. Informational.

[13] **S. Torres-Arias et al.** in-toto: Providing farm-to-table guarantees for bits and bytes. 28th USENIX Security Symposium, 2019, pp. 1393–1410.

[14] **NIST.** Secure Software Development Framework (SSDF) Version 1.1. SP 800-218, February 2022.

[15] **SLSA.** Build: Requirements for producing artifacts. SLSA specification v1.2, approved 24 November 2025.

[16] **GitHub.** npm trusted publishing with OIDC is generally available. GitHub Changelog, 31 July 2025.

[17] **Sigstore.** Verifying Signatures. Cosign documentation.

[18] **SPDX Project.** SPDX Overview. Software bill of materials specification overview.

[19] **Sigstore.** Rekor. Transparency log documentation.

[20] **IETF (B. Laurie, E. Messeri, R. Stradling).** Certificate Transparency Version 2.0. RFC 9162, December 2021. Experimental.

[21] **IETF (H. Birkholz et al.).** An Architecture for Trustworthy and Transparent Digital Supply Chains. RFC 9943, June 2026. Proposed Standard.

[22] **The Update Framework.** The Update Framework specification. Maintained specification.

[23] **SPIFFE Project.** SPIFFE Concepts. Workload identifiers, trust domains and identity documents.

[24] **SPIFFE Project.** SPIRE Concepts. Node attestation, workload attestation and registration.

[25] **NIST (D. Temoshok et al.).** Digital Identity Guidelines: Authentication and Authenticator Management. SP 800-63B-4, July 2025.

[26] **IETF.** OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens. RFC 8705, February 2020. Standards Track.

[27] **IETF.** OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449, September 2023. Standards Track.

[28] **IETF (M. Jones et al.).** OAuth 2.0 Token Exchange. RFC 8693, January 2020. Standards Track.

[29] **IETF (B. Campbell, J. Bradley, H. Tschofenig).** Resource Indicators for OAuth 2.0. RFC 8707, February 2020. Standards Track.

[30] **IETF (T. Lodderstedt, J. Richer, B. Campbell).** OAuth 2.0 Rich Authorization Requests. RFC 9396, May 2023. Standards Track.

[31] **CA/Browser Forum.** Ballot SC081v3: Introduce Schedule of Reducing Validity and Data Reuse Periods. Adopted April 2025.

[32] **CA/Browser Forum.** Baseline Requirements for the Issuance and Management of Publicly-Trusted TLS Server Certificates. Version 2.3.1, 4 October 2026, §6.3.2, and earlier versions.

[33] **Let's Encrypt (J. Aas).** OCSP Service Has Reached End of Life. 6 August 2025.

[34] **Let's Encrypt (M. McPherrin).** 6-day and IP Address Certificates are Generally Available. 15 January 2026.

[35] **IETF (L. Lundblade et al.).** The Entity Attestation Token (EAT). RFC 9711, April 2025. Standards Track.

[36] **Keylime Project.** IMA / Integrity Measurement Architecture. Runtime-integrity documentation, version 7.11.0.

[37] **M. Ruffin, C. Wang, G. Almasi, A. Adebayo, H. Franke, G. Wang.** Towards Continuous Integrity Attestation and Its Challenges in Practice: A Case Study of Keylime. IEEE/IFIP DSN 2025, pp. 256–265.

[38] **AMD / J. Powell.** AMD SEV-SNP Attestation: Establishing Trust in Guests. Linux Security Summit Europe, September 2022.

[39] **Intel.** Intel Trust Domain Extensions (Intel TDX). Architecture and developer overview.

[40] **Arm.** Arm Confidential Compute Architecture. Architecture overview: Realm Management Extension and Realms.

[41] **Confidential Containers Project.** Attestation Architecture. Trustee, attestation service and key-broker architecture.

[42] **Confidential Containers Project.** Securing Your Workload. Runtime selection, guest policy and attested Init-Data.

[43] **Docker.** Docker Engine security. Daemon attack surface and container security documentation.

[44] **OWASP Cheat Sheet Series.** Docker Security Cheat Sheet. Rule 1. Accessed 24 October 2026.

[45] **gVisor Project.** Security Model. Sandbox threat model and isolation boundaries.

[46] **A. Agache et al.** Firecracker: Lightweight Virtualization for Serverless Applications. 17th USENIX NSDI, 2020, pp. 419–434.

[47] **Cilium Project.** Threat Model. Security assumptions and cluster trust boundaries.

[48] **PostgreSQL Global Development Group.** Row Security Policies. PostgreSQL documentation, current edition.

[49] **CISA and NSA.** Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development. Cybersecurity information sheet, 24 June 2025.

[50] **OWASP GenAI Security Project.** OWASP Top 10 for LLM Applications 2025. November 2024. LLM01 Prompt Injection; LLM06 Excessive Agency.

[51] **NIST (A. Vassilev et al.).** Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. NIST AI 100-2 E2025, March 2025, §3.4.

[52] **S. Willison.** The lethal trifecta for AI agents: private data, untrusted content, and external communication. 16 June 2025.

[53] **E. Debenedetti et al.** Defeating Prompt Injections by Design. IEEE SaTML 2026, pp. 587–618. arXiv:2503.18813.

[54] **Model Context Protocol.** Authorization. Specification revision 2026-07-28.

[55] **Model Context Protocol.** Security Best Practices. Revision 2026-07-28.

[56] **IETF (M.B. Jones, P. Hunt, A. Parecki).** OAuth 2.0 Protected Resource Metadata. RFC 9728, April 2025. Standards Track.

[57] **NIST.** Protecting the Integrity of Internet Routing: Border Gateway Protocol (BGP) Route Origin Validation. SP 1800-14, June 2019.

[58] **IETF.** BGP Prefix Origin Validation. RFC 6811, January 2013. Standards Track.

[59] **IETF.** Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing. RFC 2827 / BCP 38, May 2000.

[60] **IETF.** Distributed Denial-of-Service Open Threat Signaling (DOTS) Signal Channel Specification. RFC 9132, September 2021. Standards Track.

[61] **D. Adrian, D. Benjamin, B. Beck, D. O'Brien (Google).** A new path for Kyber on the web. Google Online Security Blog, 13 September 2024.

[62] **Cloudflare.** Cloudflare targets 2029 for full post-quantum security. 7 April 2026.

[63] **J. De Meulemeester, L. Wilke, D. Oswald, T. Eisenbarth, I. Verbauwhede, J. Van Bulck.** BadRAM: Practical Memory Aliasing Attacks on Trusted Execution Environments. IEEE S&P 2025, pp. 4117–4135. CVE-2024-21944.

[64] **J. De Meulemeester, D. Oswald, I. Verbauwhede, J. Van Bulck.** Battering RAM: Low-Cost Interposer Attacks on Confidential Computing via Dynamic Memory Aliasing. IEEE S&P 2026, pp. 4391–4407.

[65] **AMD.** SEV-SNP Physical Memory Aliasing. Security bulletin AMD-SB-3024, 30 September 2025.

[66] **A. Seto, O. K. Duran, S. Amer, J. Chuang, S. van Schaik, D. Genkin, C. Garman.** WireTap: Breaking Server SGX via DRAM Bus Interposition. ACM CCS 2025, pp. 708–722.

[67] **J. Chuang, A. Seto, N. Berrios, S. van Schaik, C. Garman, D. Genkin.** TEE.Fail: Breaking Trusted Execution Environments via DDR5 Memory Bus Interposition. IEEE S&P 2026, pp. 4408–4426.

[68] **J. De Meulemeester et al.** DDRop: Generic Memory Interposer Attacks on Confidential VMs by Dropping DDR5 Writes. ACM CCS 2026; disclosed September 2026.

[69] **E. Apsey, P. Rogers, M. O'Connor, R. Nertney (NVIDIA).** Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI. NVIDIA Technical Blog, 3 August 2023.

[70] **NVIDIA.** BlueField Modes of Operation. DOCA 3.5 archive. DPU and restricted host modes.

[71] **University of Cambridge.** CHERI: Capability Hardware Enhanced RISC Instructions. Research overview and capability-system design.

[72] **NIST.** Platform Firmware Resiliency Guidelines. SP 800-193, May 2018.

[73] **AMD.** AMD SEV Confidential Computing Vulnerability. Security bulletin AMD-SB-3019 (CVE-2024-56161), 3 February 2025.

[74] **NIST.** Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, August 2024.

[75] **NIST.** Module-Lattice-Based Digital Signature Standard. FIPS 204, August 2024.

[76] **NIST.** Stateless Hash-Based Digital Signature Standard. FIPS 205, August 2024.

[77] **NIST (D. Moody et al.).** Transition to Post-Quantum Cryptography Standards. NIST IR 8547, initial public draft, November 2024.

[78] **NIST (D. Cooper et al.).** Recommendation for Stateful Hash-Based Signature Schemes. SP 800-208, October 2020.

[79] **NSA.** Announcing the Commercial National Security Algorithm Suite 2.0. Cybersecurity advisory, September 2022.

[80] **IETF.** JSON Canonicalization Scheme (JCS). RFC 8785, June 2020. Informational.

[81] **Open Policy Agent.** Bundles. Policy distribution and bundle-signature verification documentation.

[82] **IETF (D. Cooper et al.).** Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280, May 2008, §6.1.3.

[83] **Envoy Project.** External Authorization. HTTP authorization filter documentation.

[84] **CISA.** StopRansomware Guide. Prevention, incident response and recovery guidance.

[85] **Amazon Web Services.** Locking objects with Object Lock. S3 retention modes and write-once-read-many controls.

Colophon. Set in Source Serif 4, Inter and JetBrains Mono. Every figure and both artworks are generated from code (Python to SVG) and typeset with the text in Chromium; formulas by KaTeX. The chart palette was checked for colour-vision deficiency. **Artwork:** *Bounded Weave* traces streamlines of a uniform flow past a circular boundary. The flow is potential flow around a cylinder, so no streamline crosses the circle. Inside, point vortices churn without escaping, held in by mirror-image vortices placed outside the circle: containment enforced from outside, as in design rule 4. Each thread is drawn as a run of short dashes with small gaps: authority issued as renewable leases. Seeds 2026 (cover) and 1024 (closing). Source, build scripts and an interactive version of the artwork sit beside the PDF.